



ARCHITECTURE (Frontend + Backend + AI/ML + Solver)

Version: 1.0 — SIH Production Blueprint

Scope: End-to-end system from learner's browser to hash-chained audit ledger to AI provider fallback chain to MILP solver fleet.

Design Authority: Principal Distributed Systems Architect

0. ARCHITECTURAL PRINCIPLES (The Constitution)

Before any diagram, these are non-negotiable. Every component must honor them.

#	Principle	Enforcement
1	AI proposes, rules decide	AI returns structured scores. Deterministic recognizer classifies. Human approves.
2	Human is a structural gate, not a feature	Graph builder consumes only VALIDATED events. No auto-promotion.
3	Every decision is	DecisionBundle

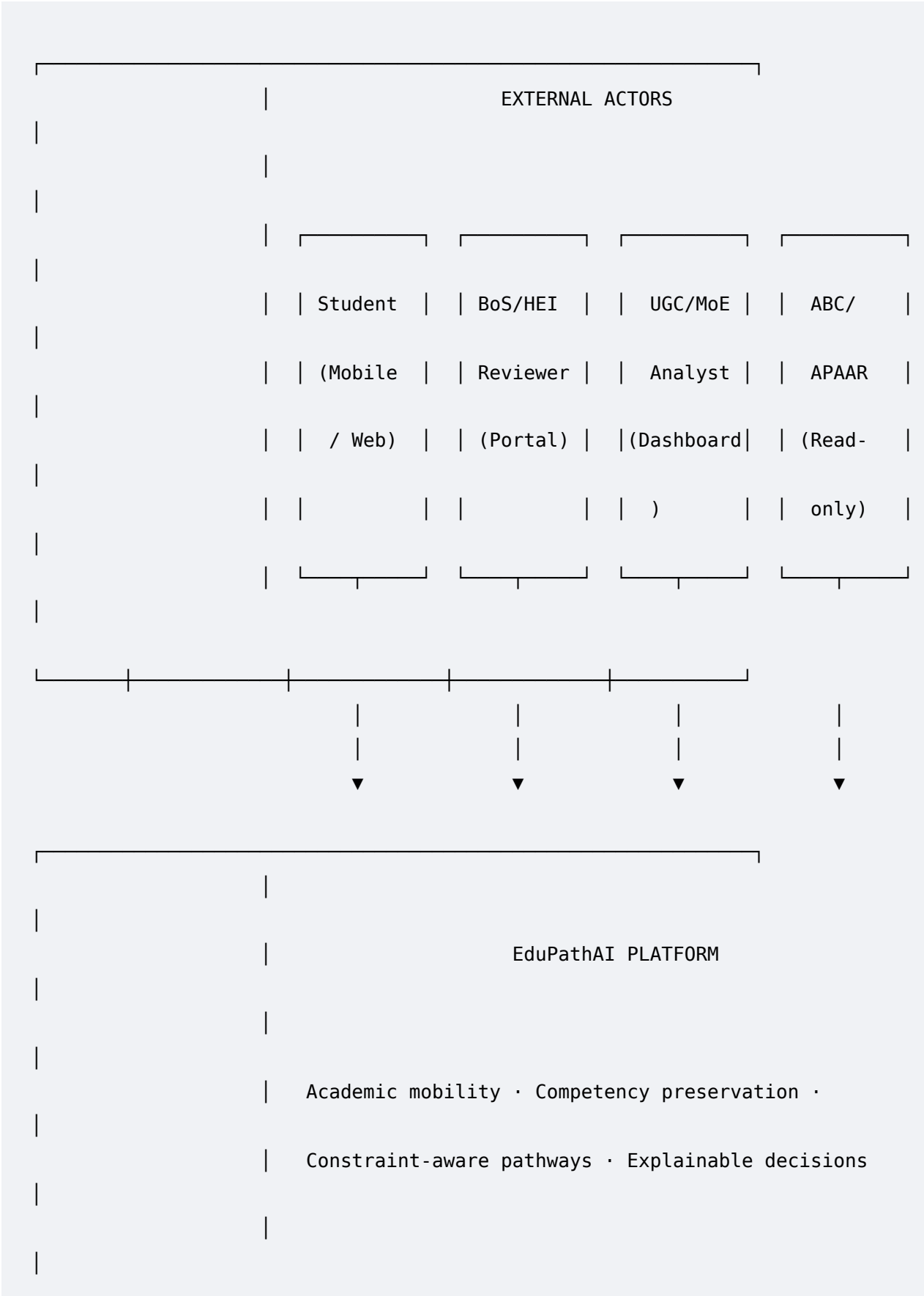
	reproducibl e	captures 13 versions. Snapshot at request time.
4	Every decision is tamper- evident	Partitioned SHA-256 hash chain. KMS- envelope keys. S3 Object Lock.
5	Every failure degrades to REVIEW	Provider down → fallback → REVIEW. Solver timeout → HEURISTI C. Never silent wrong.
6	Every component is replaceable	Model- agnostic AI Gateway. Resource Registry. Institution Adapter.
7	Evidence first,	Matching returns

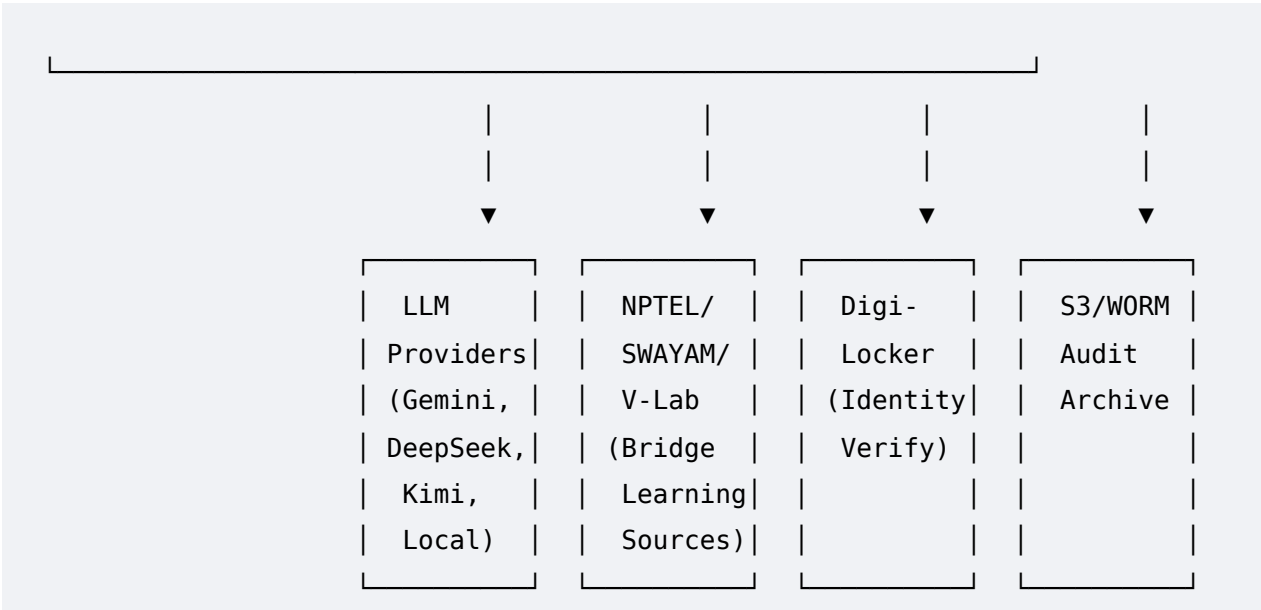
	optimization second	evidence with page numbers. MILP consumes evidence, not opinions.
8	Trace $\neq$ Decision $\neq$ Audit	Three distinct IDs, linked. Contestability requires exact replay.
9	Nothing is claimed without derivation	Throughput, fleet size, cache hit rate — all shown work.
10	Operational simplicity over production theater	No Kafka at MVP. No Neo4j. No ClickHouse. PostgreSQL + outbox + poller.

1. C4 LEVEL 1 — SYSTEM CONTEXT

Who uses EduPathAI and what does it integrate with?

text

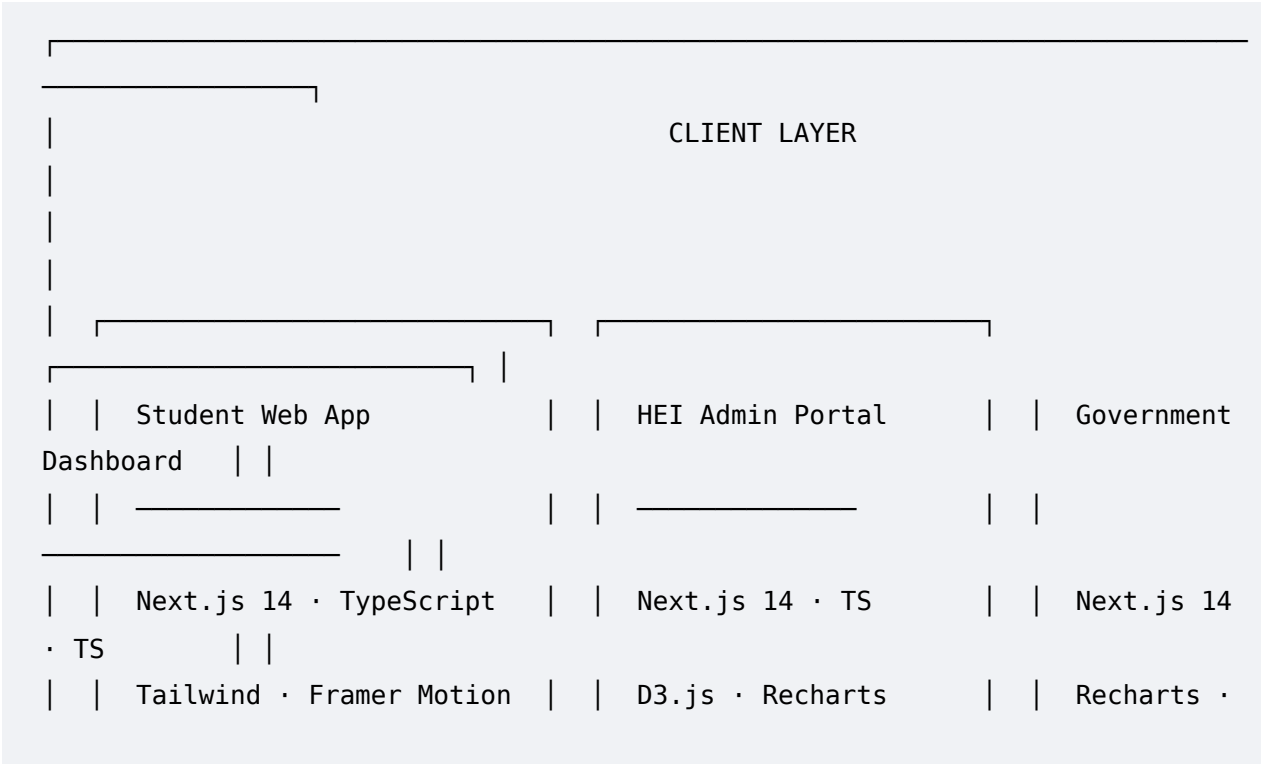


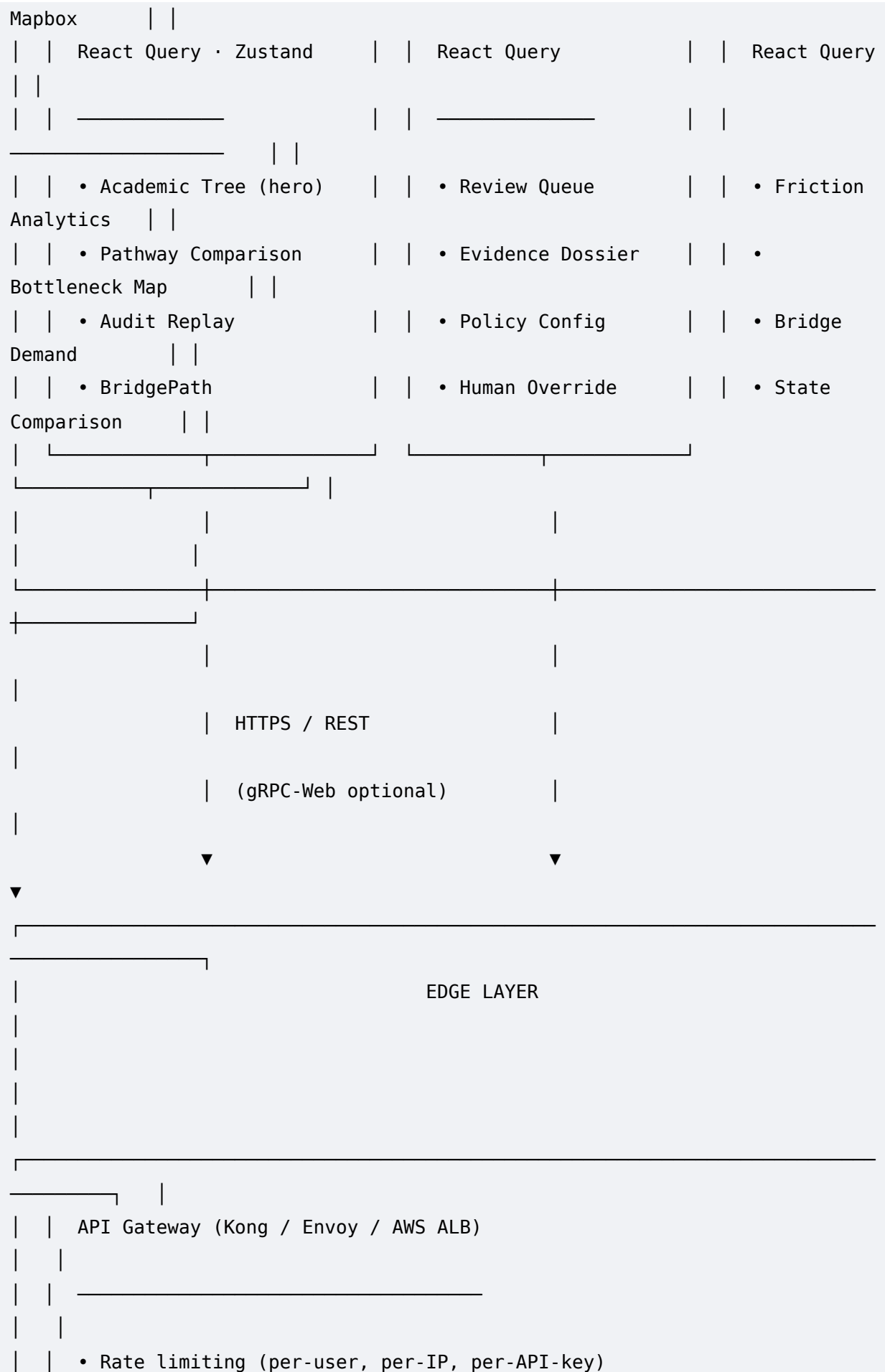


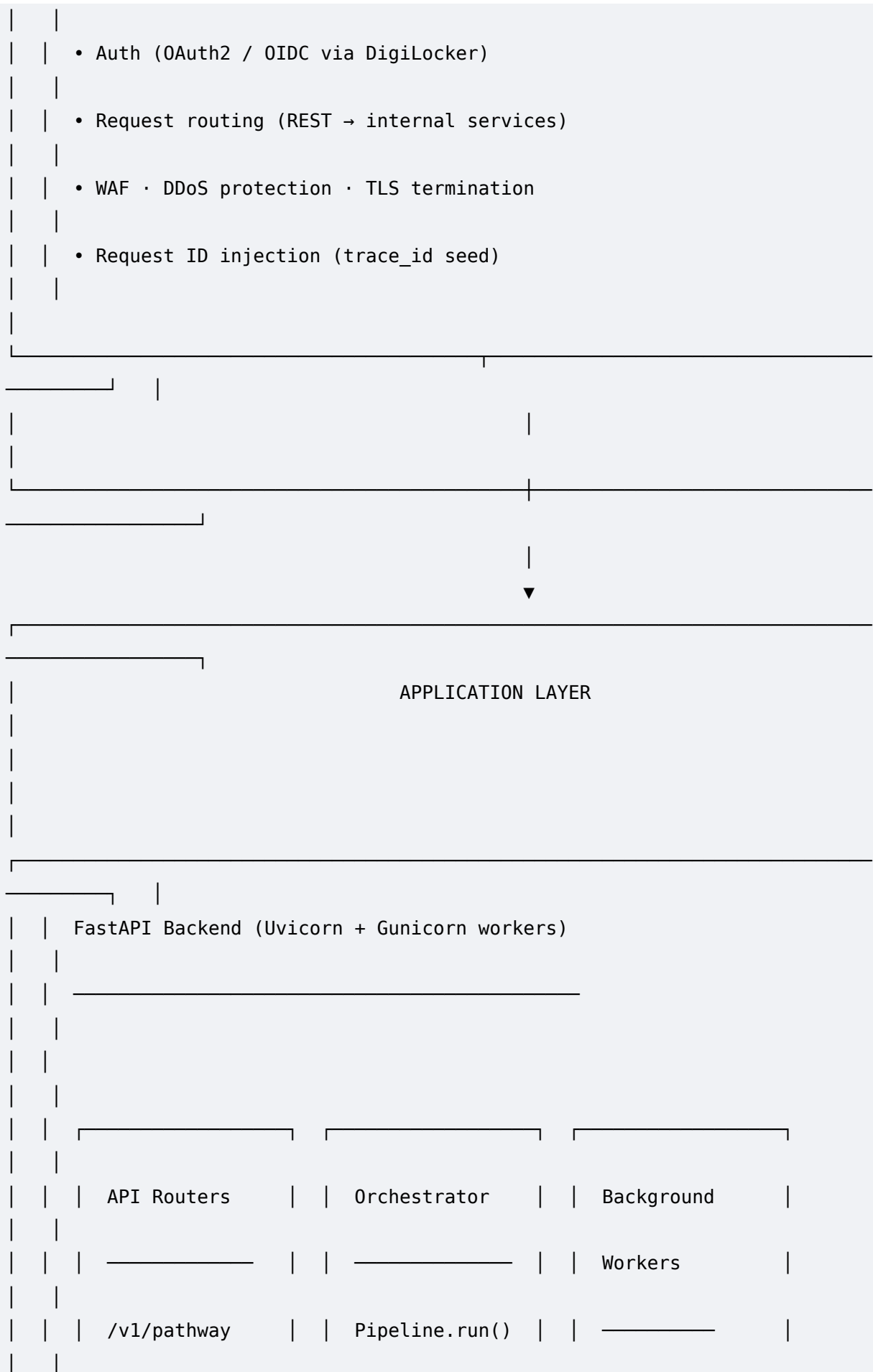
- External dependencies:
- LLM Providers: Gemini (primary), DeepSeek (secondary), Kimi (tertiary), Local Llama 3.2 (fallback)
 - Learning Resources: NPTEL, SWAYAM, Virtual Labs, HEI LMS
 - Identity: DigiLocker (read-only), ABC/APAAR (read-only)
 - Storage: S3 with Object Lock (WORM) for audit archive
- EduPathAI never writes to national infrastructure. Read-only integration.

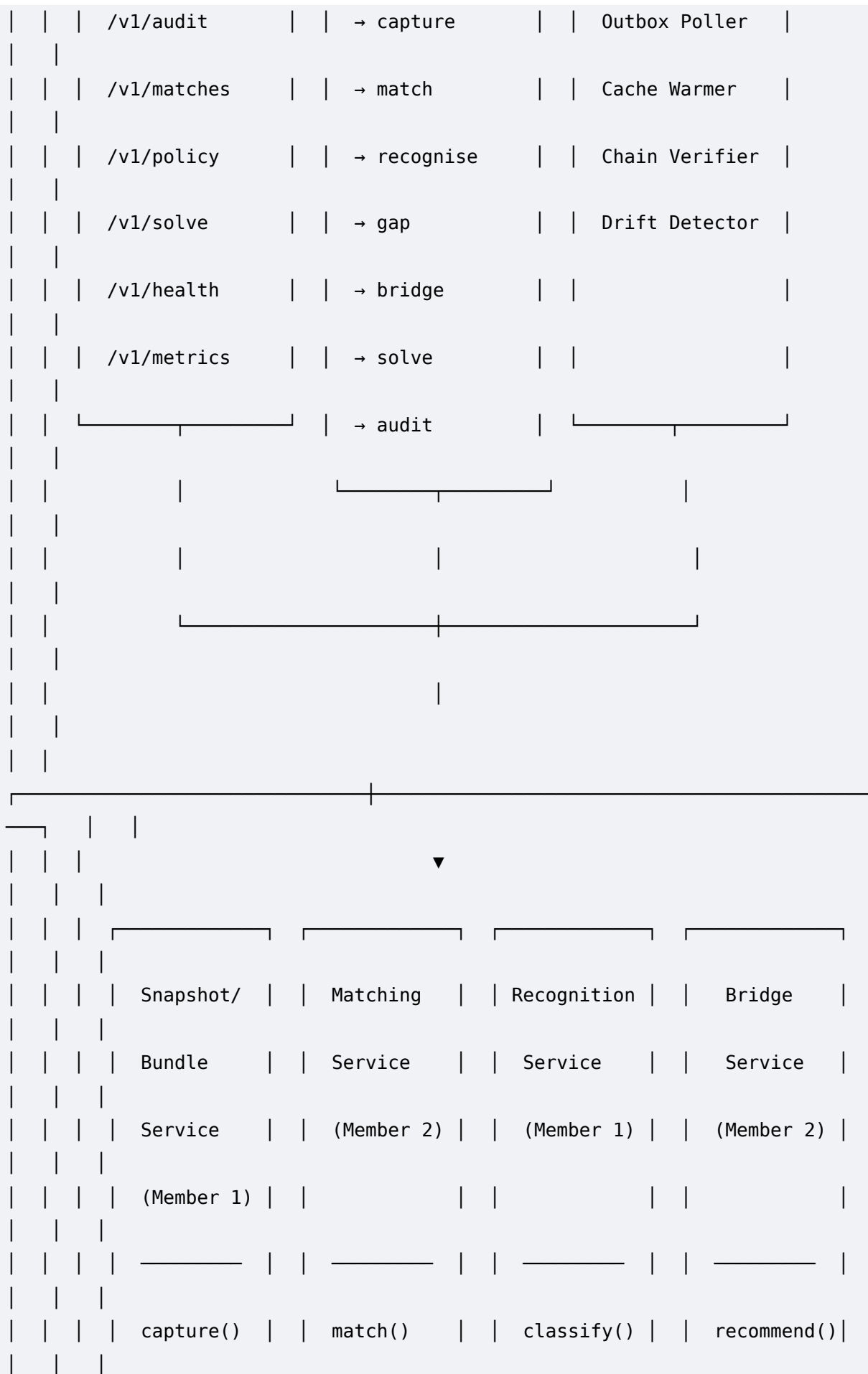
2. C4 LEVEL 2 — CONTAINER DIAGRAM (The Full Stack)

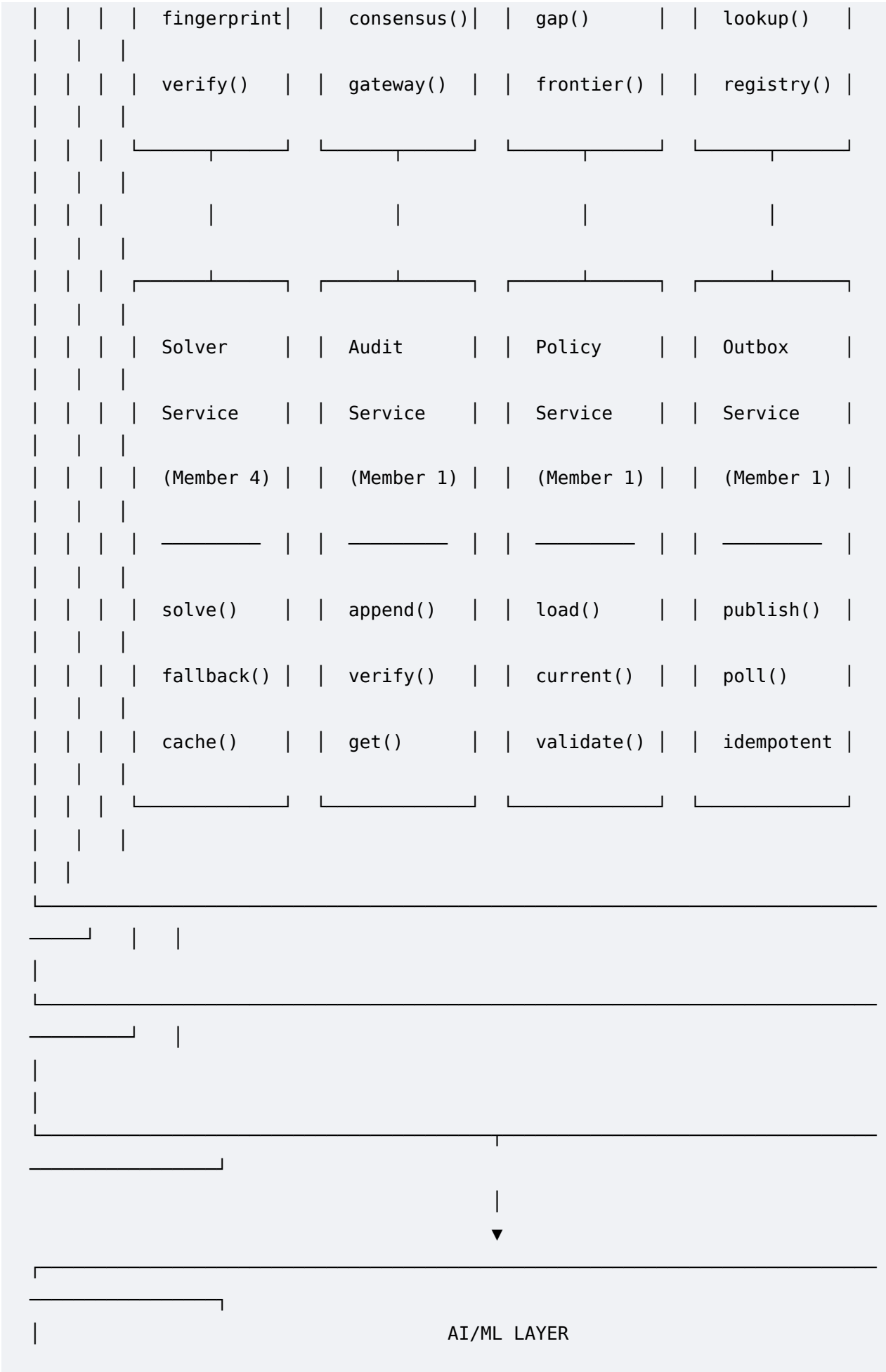
text

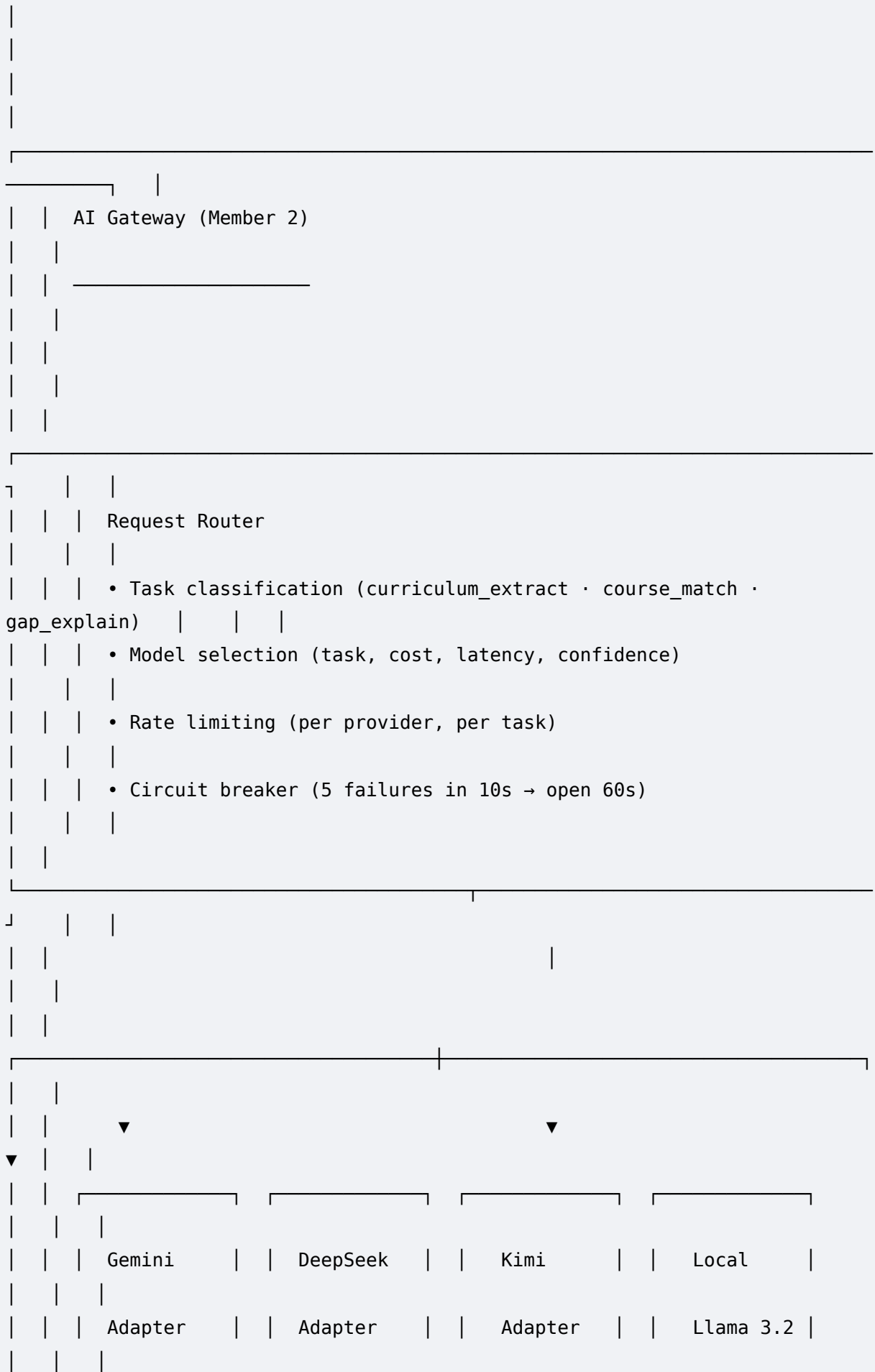


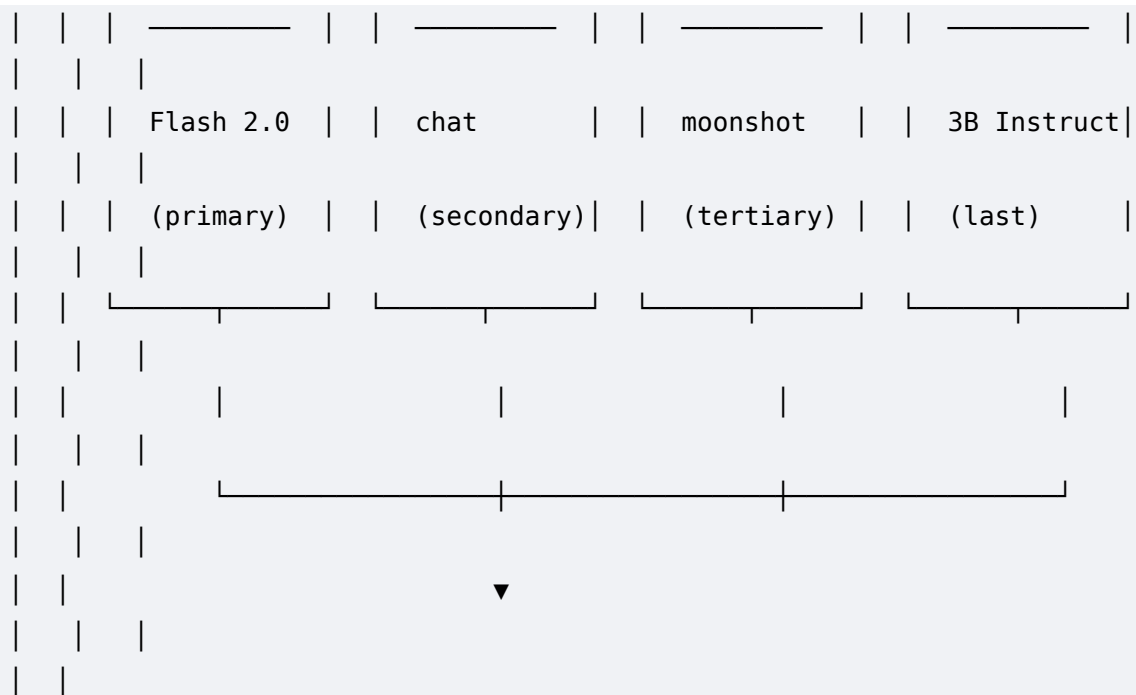












-
- Response Validator
- Schema validation (JSON Schema)
 - Confidence threshold check
 - Cross-model consensus (decision-level, not scalar)
 - Fallback to REVIEW if confidence < threshold
-

Cache Layer

-
- Cache Layer
- Exact-match (SHA-256 of input + model + prompt version)
 - Semantic cache (embedding cosine > 0.95)

• TTL: 7d (curriculum), 1d (explanations)

Matching Engine (Member 2)

Recognizer (Member 1)

• pgvector retrieval

• Deterministic classify

• Domain-sharded search

• DIRECT · BRIDGE

• Cross-encoder reranking

• MISSING · REVIEW

• Bloom taxonomy compare

• POLICY_CONFLICT

• Outcome coverage

• No AI. No LLM.

• Decision consensus

• Pure rules.

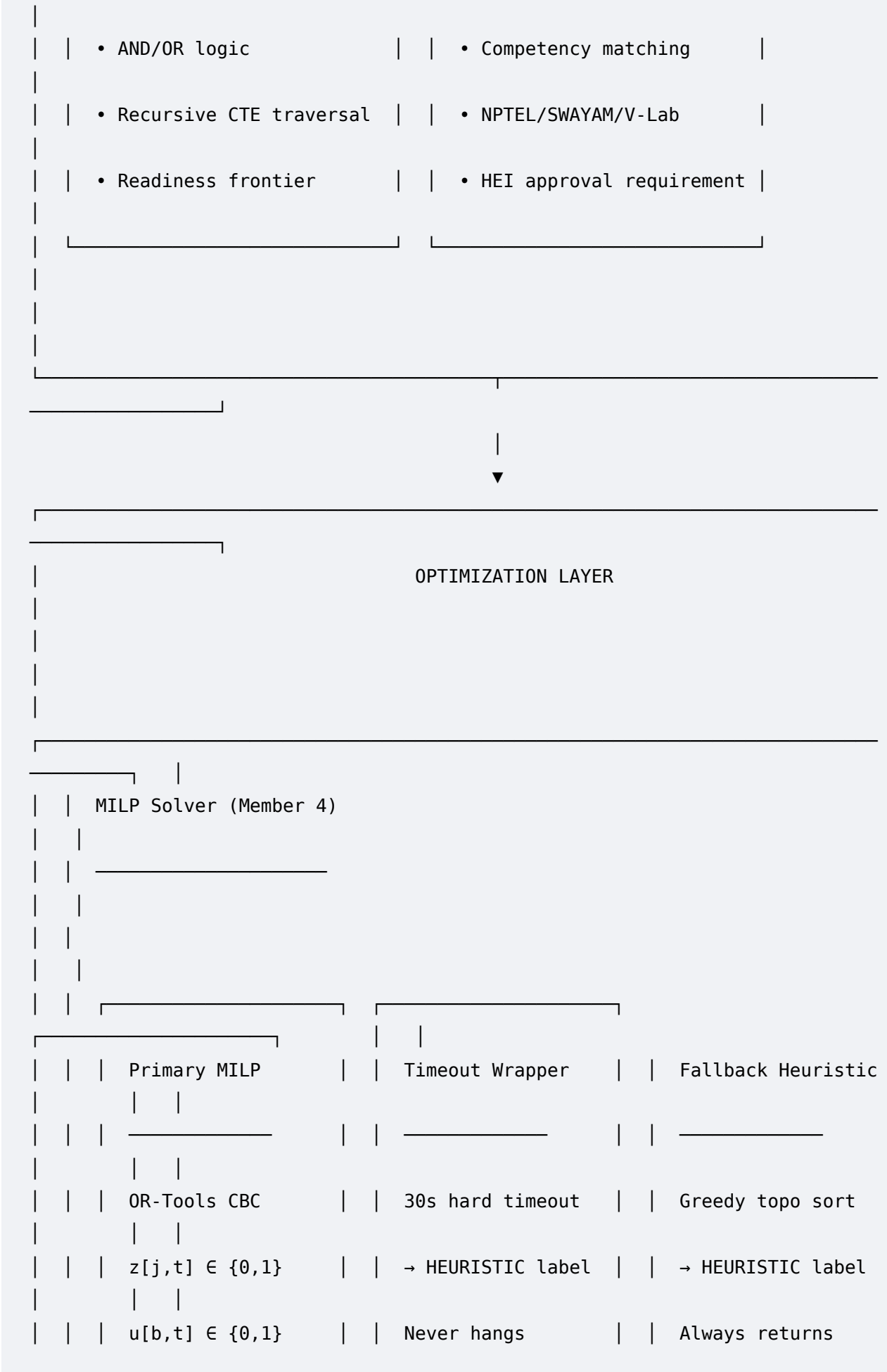


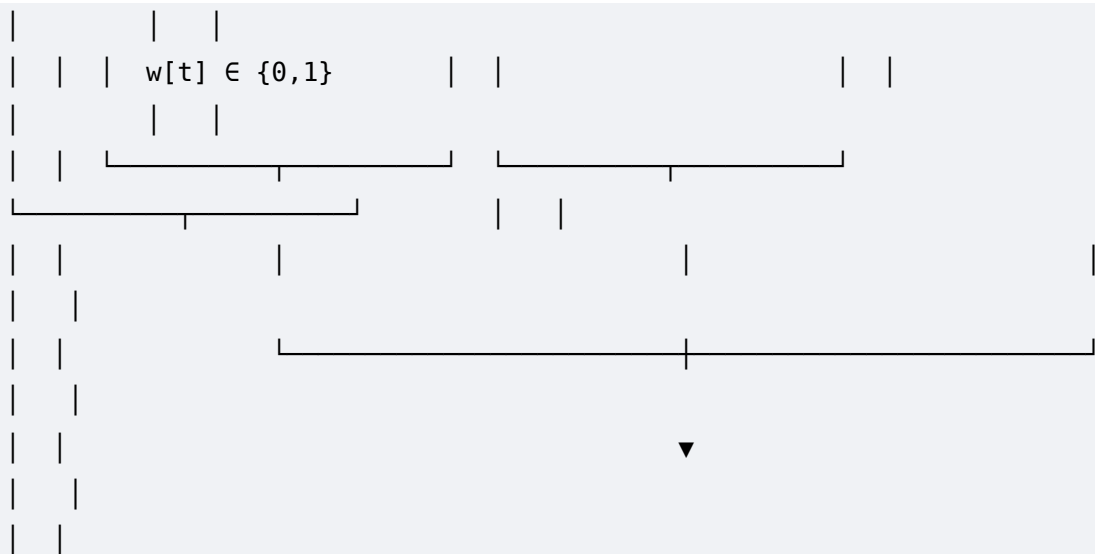
Prerequisite Hypergraph

BridgePath (Member 2)

(Member 1)

• Resource Registry





```

┌   |   |
|   |   | Precomputation Cache
|   |   |
|   |   | Key: (student_state_hash, curriculum_version, policy_version)
|   |   |
|   |   | Target hit rate: > 85%
|   |   |
|   |   | Collapses thundering herd before solver sees it
|   |   |
|   |   |
└   |   |

```

```

└   |   |
|   |
|   |
|   |

```

```

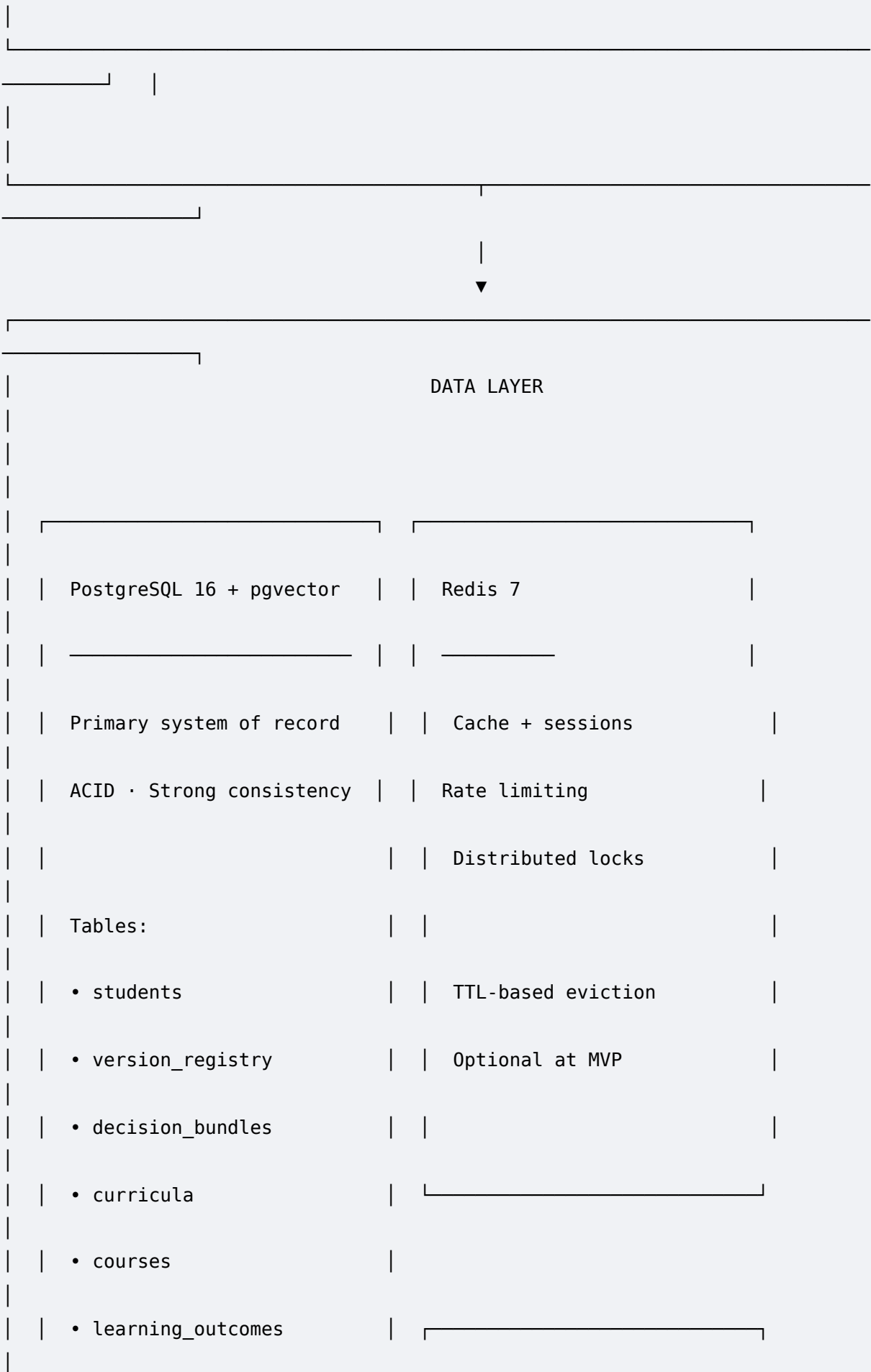
┌   |   |
|   |   | Worker Fleet
|   |   |
|   |   | • 16 base workers (Erlang-C derived)
|   |   |
|   |   | • Autoscale to 64 on queue depth > 500
|   |   |
|   |   | • Queue with position notification beyond 64
|   |   |
|   |   |
└   |   |

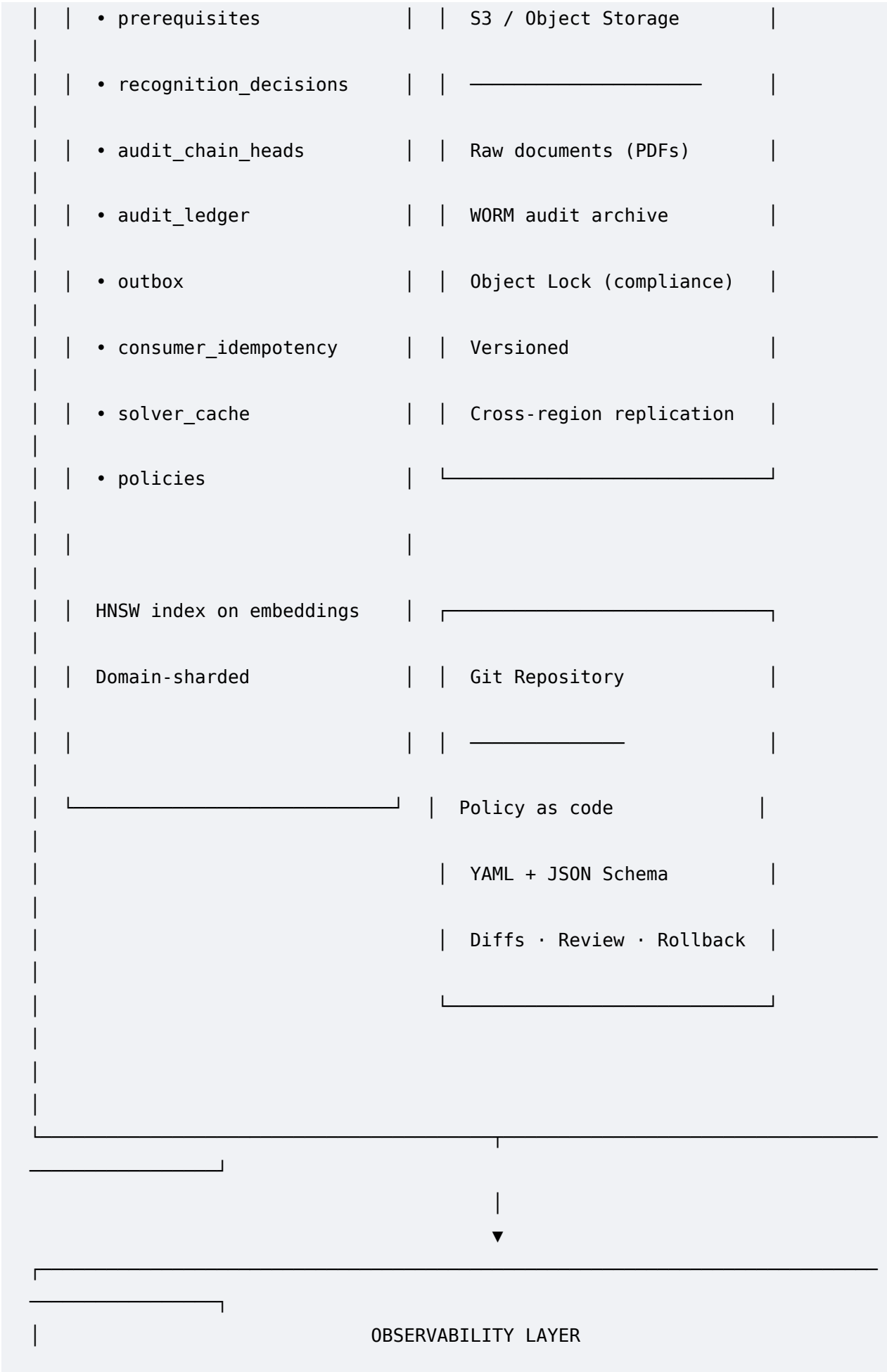
```

```

└   |   |

```

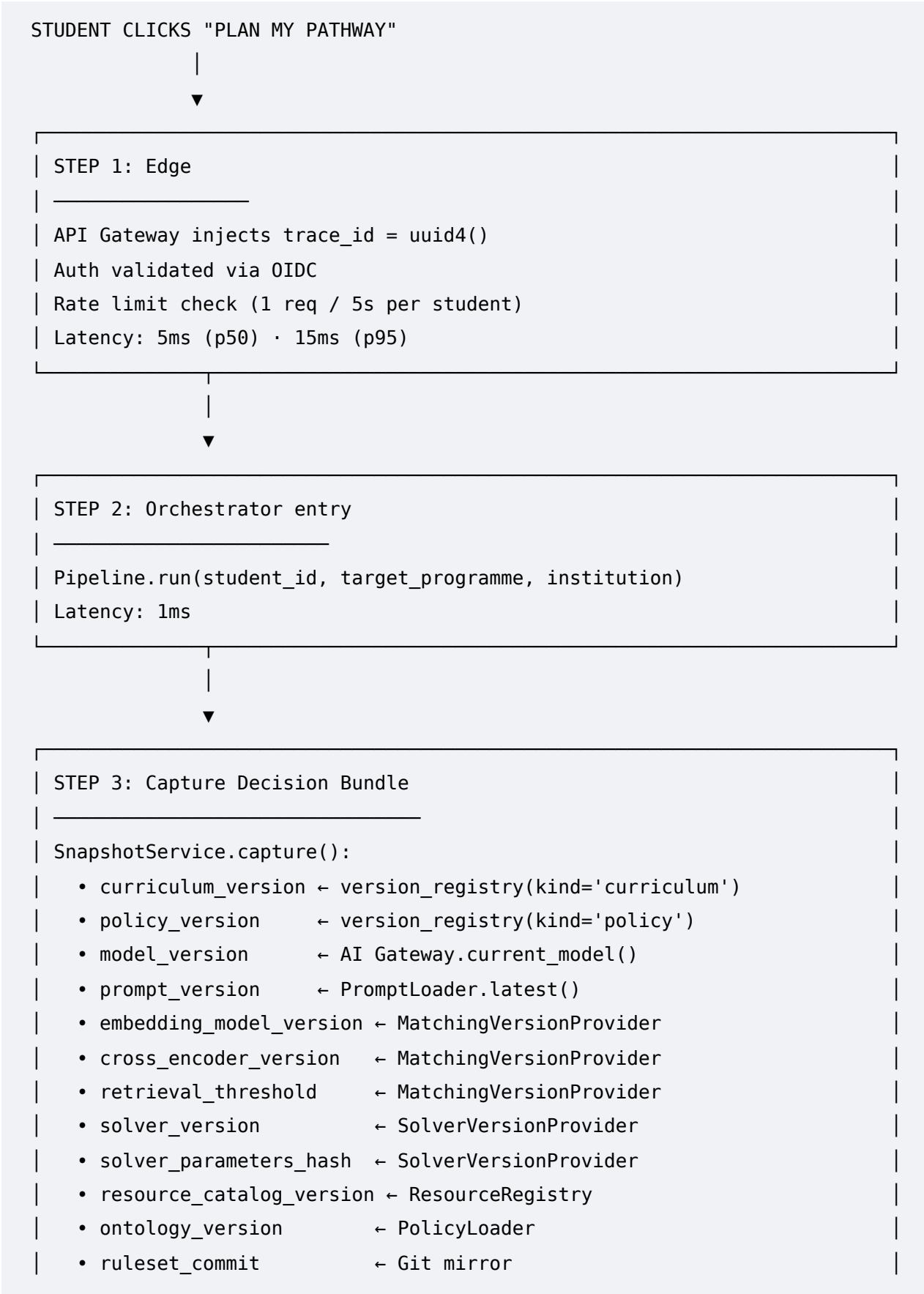




3. END-TO-END DATA FLOW (The Decision Loop)

One request, every hop, with latency budget.

text



```
| • tool_definitions_hash ← Prompts registry  
| bundle.verify() – fails fast if any missing  
| Persists to decision_bundles table  
| Latency: 10ms (p50) · 25ms (p95)
```



```
| STEP 4: Load student + curriculum
```

```
| _____  
| Query PostgreSQL:
```

- Student completed courses (from student_courses join)
- Target curriculum courses + outcomes
- Prerequisite hypergraph

```
| Latency: 15ms (p50) · 40ms (p95)
```



```
| STEP 5: Vector retrieval (candidate generation)
```

```
| _____  
| For each source course:
```

- Embed source text
- Domain shard classification
- pgvector HNSW search (top 20)
- Cross-encoder rerank (top 5)

```
| Returns: ~20 source × 5 target = 100 candidate pairs
```

```
| Latency: 300ms (p50) · 1.5s (p95)
```



```
| STEP 6: Structured matching (AI Gateway)
```

```
| _____  
| For each candidate pair:
```

- Build CanonicalRequest
- Gateway.call() → try Gemini → DeepSeek → Kimi → Local
- Circuit breaker per provider
- Cost tracked per call
- Response validated against JSON Schema

```
| Returns: MatchResult[] with 8 fields each
```

Latency: 800ms (p50) · 3s (p95)



STEP 7: Cross-model consensus

If 2+ providers responded:

- `decision_agreement() ≥ 0.85?`
- `field_agreement() ≥ 0.80?`
- If both pass → use highest-evidence result
- If either fails → REVIEW status

Latency: 5ms



STEP 8: Deterministic recognition

For each `MatchResult`:

`classify() → RecognitionStatus`

1. Policy conflict? → `POLICY_CONFLICT`
2. Prerequisite missing? → `MISSING`
3. Domain mismatch? → `REVIEW`
4. Credit incompatible? → `REVIEW`
5. Evidence quality low? → `REVIEW`
6. Coverage ≥ 0.9 AND semantic ≥ 0.85 AND assessment ≥ 0.9 → `DIRECT`
7. Coverage ≥ 0.6 OR semantic ≥ 0.65 → `BRIDGE`
8. Else → `MISSING`

NO AI. NO LLM. Pure rules.

Latency: 2ms



STEP 9: Gap analysis + frontier

For each non-DIRECT match:

`classify_gap() → Gap (KNOWLEDGE / PREREQUISITE / ASSESSMENT / ADMIN)`

Compute Academic Readiness Frontier:

`frontier = [c for c in target if hypergraph.is_unlocked(c, completed)]`

Latency: 50ms (p50) · 200ms (p95)



STEP 10: BridgePath recommendation

For each Gap:

ResourceRegistry.find_for_outcomes(missing_outcomes)

Returns: (resource, coverage_score) sorted by coverage

Build Bridge objects with:

- resource_id · provider · url
- competency_coverage · duration_hours
- assessment_available · recognition_status

Latency: 10ms



STEP 11: MILP pathway optimization

SolverWithFallback.solve():

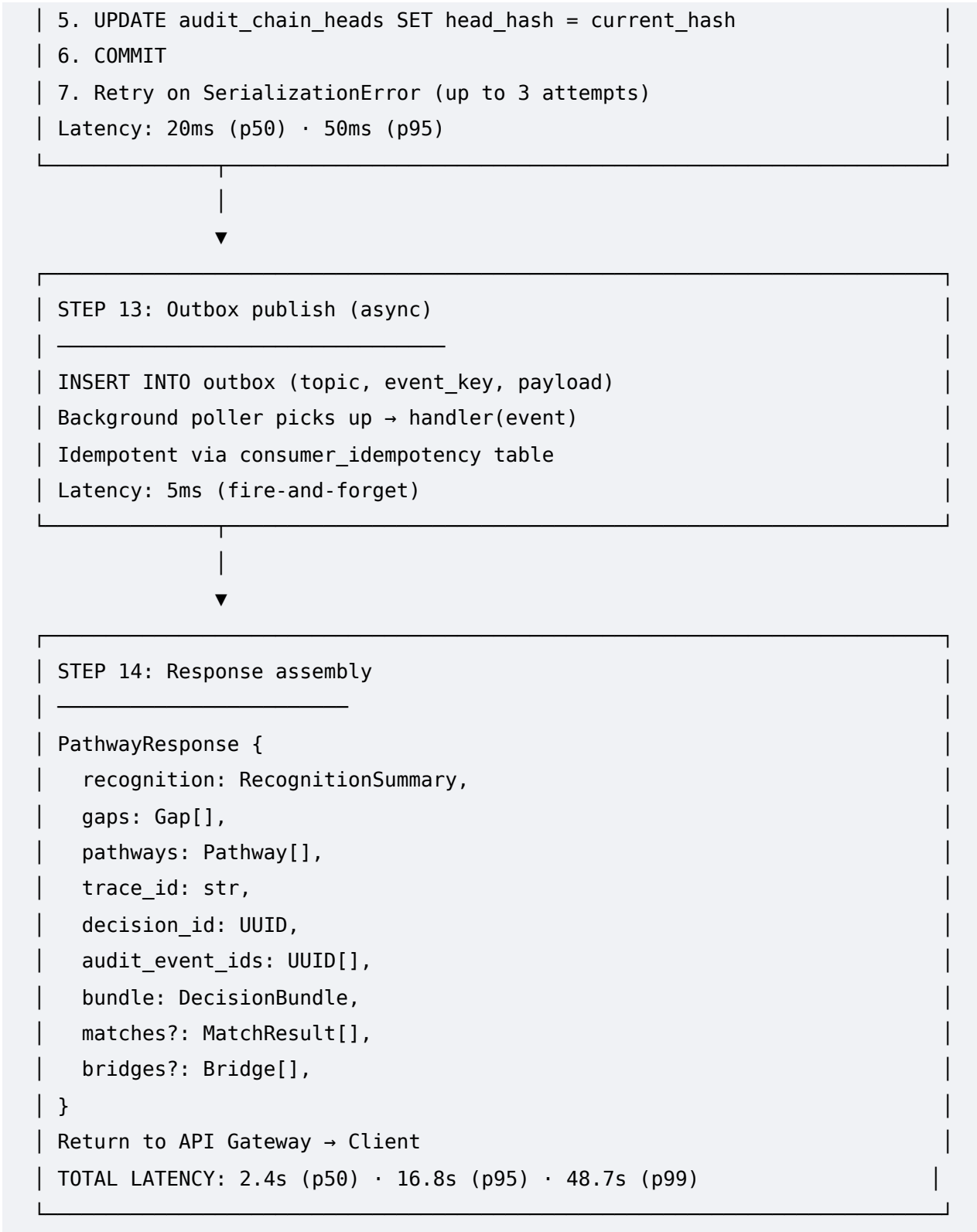
1. Check cache (key = student_state_hash + curriculum + policy)
2. If cache hit → return cached (0ms)
3. Else → MILPSolver.solve_all_modes()
 - FASTEST: minimize completion_time
 - BALANCED: balanced weights
 - MAX_PRESERVATION: maximize recognized credits
4. Hard timeout 30s → FallbackSolver.solve() (HEURISTIC label)
5. Honest status: OPTIMAL | FEASIBLE_NOT_OPTIMAL | HEURISTIC

Latency: 2s (p50) · 15s (p95) · 30s (p99 hard cutoff)



STEP 12: Audit append (partitioned hash chain)

1. BEGIN SERIALIZABLE
2. SELECT head_hash FROM audit_chain_heads WHERE chain_id=\$1 FOR UPDATE
3. Compute current_hash = sha256(previous_hash + canonical_json(record))
4. INSERT INTO audit_ledger



4. COMPONENT INTERACTION MATRIX

Who calls whom, with what protocol, and why.

From	To	Protocol	Sync/Async	Purpose
			c	

Client	API Gateway	HTTPS/REST	Sync	Request pathway
API Gateway	FastAPI	HTTP/1.1	Sync	Route request
FastAPI	Orchestrator	In-process	Sync	Execute pipeline
Orchestrator	SnapshotService	In-process	Sync	Capture bundle
SnapshotService	PostgreSQL	asyncpg	Sync	Persist bundle
Orchestrator	Matcher (Member 2)	In-process	Sync	Get matches
Matcher	VectorSearch	In-process	Sync	Retrieve candidates
VectorSearch	pgvector	asyncpg	Sync	HNSW search
Matcher	CrossEncoder	In-process	Sync	Rerank
Matcher	AIGateway	In-process	Sync	Structured scores
AIGateway	Gemini	HTTPS	Sync	Primary LLM
AIGateway	DeepSeek	HTTPS	Sync	Secondary LLM
AIGateway	Kimi	HTTPS	Sync	Tertiary LLM

AIGateway	LocalLlama	In-process	Sync	Fallback LLM
Matcher	Consensus	In-process	Sync	Decision agreement
Orchestrator	Recognizer	In-process	Sync	Classify
Orchestrator	BridgePath	In-process	Sync	Resource lookup
BridgePath	ResourceRegistry	In-process	Sync	Query catalog
Orchestrator	SolverWith Fallback	In-process	Async	MILP
SolverWith Fallback	SolverCache	asyncpg	Sync	Cache check
SolverWith Fallback	MILPSolver	In-process	Sync	OR-Tools
SolverWith Fallback	FallbackSolver	In-process	Sync	Greedy heuristic
Orchestrator	Ledger	asyncpg	Sync	Append audit
Ledger	PostgreSQL	asyncpg	Sync	Hash chain
Orchestrator	Outbox	asyncpg	Sync	Publish event
OutboxPoller	EventHandlers	In-process	Async	Process event


```

|
| _____
|
| id                UUID PK
|
| kind              TEXT ('curriculum'|'policy'|'model'|'prompt'|'embedding'|
|
|                   'cross_encoder'|'solver'|'resource'|'ontology')
|
| scope_key         TEXT ('BTech-CSE' | 'IIT-Bombay/BTech-CSE')
|
| version           TEXT
|
| is_current        BOOLEAN
|
| published_at      TIMESTAMPTZ
|
| UNIQUE (kind, scope_key, version)
|
| Partial unique index: (kind, scope_key) WHERE is_current = TRUE
|
|_____
|
└

```

```

|_____
└
| decision_bundles
|
| _____
|
| id                UUID PK
|
| curriculum_version TEXT NOT NULL
|
| policy_version    TEXT NOT NULL
|
| model_version     TEXT NOT NULL
|
| prompt_version    TEXT NOT NULL
|
| embedding_model_version TEXT NOT NULL

```



```
|
| validated_at    TIMESTAMPTZ
|
| validated_by    TEXT
|
| UNIQUE (institution, programme, version)
|
```

└─

```
|
└─
| courses
|
| _____
|
| id              UUID PK
|
| curriculum_id   UUID FK → curricula
|
| code            TEXT
|
| name            TEXT
|
| credits          NUMERIC(4,2) CHECK > 0
|
| semester        INT
|
| modality         TEXT ('theory' | 'lab' | 'theory+lab')
|
| UNIQUE (curriculum_id, code)
|
```

└─

```
|
└─
| learning_outcomes
|
| _____
|
```

```
| id          UUID PK
|
| course_id   UUID FK → courses
|
| outcome_text TEXT
|
| bloom_level INT CHECK 1..6
|
| domain      TEXT
|
| embedding    VECTOR(384) (all-MiniLM-L6-v2)
|
| UNIQUE (course_id, outcome_text)
|
| HNSW index: embedding vector_cosine_ops
|
```

```
| prerequisites
|
| _____
|
| id          UUID PK
|
| course_id   UUID FK → courses
|
| hyperedge_id UUID (groups AND-conditions)
|
| required_course UUID FK → courses
|
| logic        TEXT ('AND'|'OR')
|
| UNIQUE (course_id, hyperedge_id, required_course)
|
```

┌	
	recognition_decisions

	id UUID PK
	student_id UUID FK → students
	source_course_id UUID FK → courses
	target_course_id UUID FK → courses
	status TEXT CHECK
	(DIRECT BRIDGE MISSING REVIEW POLICY_CONFLICT)
	confidence NUMERIC(5,4) CHECK 0..1
	bundle_id UUID FK → decision_bundles
	evidence_refs JSONB
	missing_outcomes JSONB
	created_at TIMESTAMPTZ
└	
┌	

┌	
	audit_chain_heads

	chain_id TEXT PK ('global' 'institution:X' 'daily:2026-09-11')
	head_hash TEXT NOT NULL
	head_record_id UUID
	updated_at TIMESTAMPTZ
└	

| Seed: ('global', 'GENESIS')

| audit_ledger

| id UUID PK

| chain_id TEXT FK → audit_chain_heads

| recommendation_id UUID

| decision_id UUID (distinct from trace)

| previous_hash TEXT

| current_hash TEXT

| bundle_id UUID FK → decision_bundles

| input_hash TEXT

| output_hash TEXT

| confidence NUMERIC(5,4)

| evidence JSONB

| ai_recommendation TEXT

| human_decision TEXT

| trace_id TEXT (OTel trace)

| timestamp TIMESTAMPTZ

```
|
| UNIQUE (chain_id, current_hash)
|
└─
```

```
└─
| outbox
|
| ───
```

```
| id          UUID PK
```

```
| topic       TEXT
```

```
| event_key    TEXT (for idempotency)
```

```
| payload      JSONB
```

```
| created_at   TIMESTAMPTZ
```

```
| published_at TIMESTAMPTZ
```

```
| UNIQUE (topic, event_key)
|
└─
```

```
└─
| consumer_idempotency
|
| ───────────
```

```
| consumer     TEXT
```

```
| event_key    TEXT
```

```
| processed_at TIMESTAMPTZ
|
```

| PRIMARY KEY (consumer, event_key)

|

|

|

| solver_cache

|

| _____

|

| cache_key TEXT PK

|

| response JSONB

|

| created_at TIMESTAMPTZ

|

| expires_at TIMESTAMPTZ

|

|

|

| policies

|

| _____

|

| id UUID PK

|

| institution TEXT

|

| programme TEXT

|

| version TEXT

|

| effective_from DATE

|

| effective_to DATE

|

| git_commit_hash TEXT

```
|
| content          JSONB
|
| UNIQUE (institution, programme, version)
|
└─
```

5.2 Redis Keys (Optional at MVP)

TEXT

```
session:{user_id}           → JSON (TTL 24h)
rate_limit:{user_id}:{endpoint} → counter (TTL 60s)
lock:outbox:poller          → distributed lock (TTL 30s)
cache:curriculum:{version}   → JSON (TTL 7d)
cache:policy:{institution}:{prog} → JSON (TTL 1h)
```

5.3 S3 Bucket Structure

TEXT

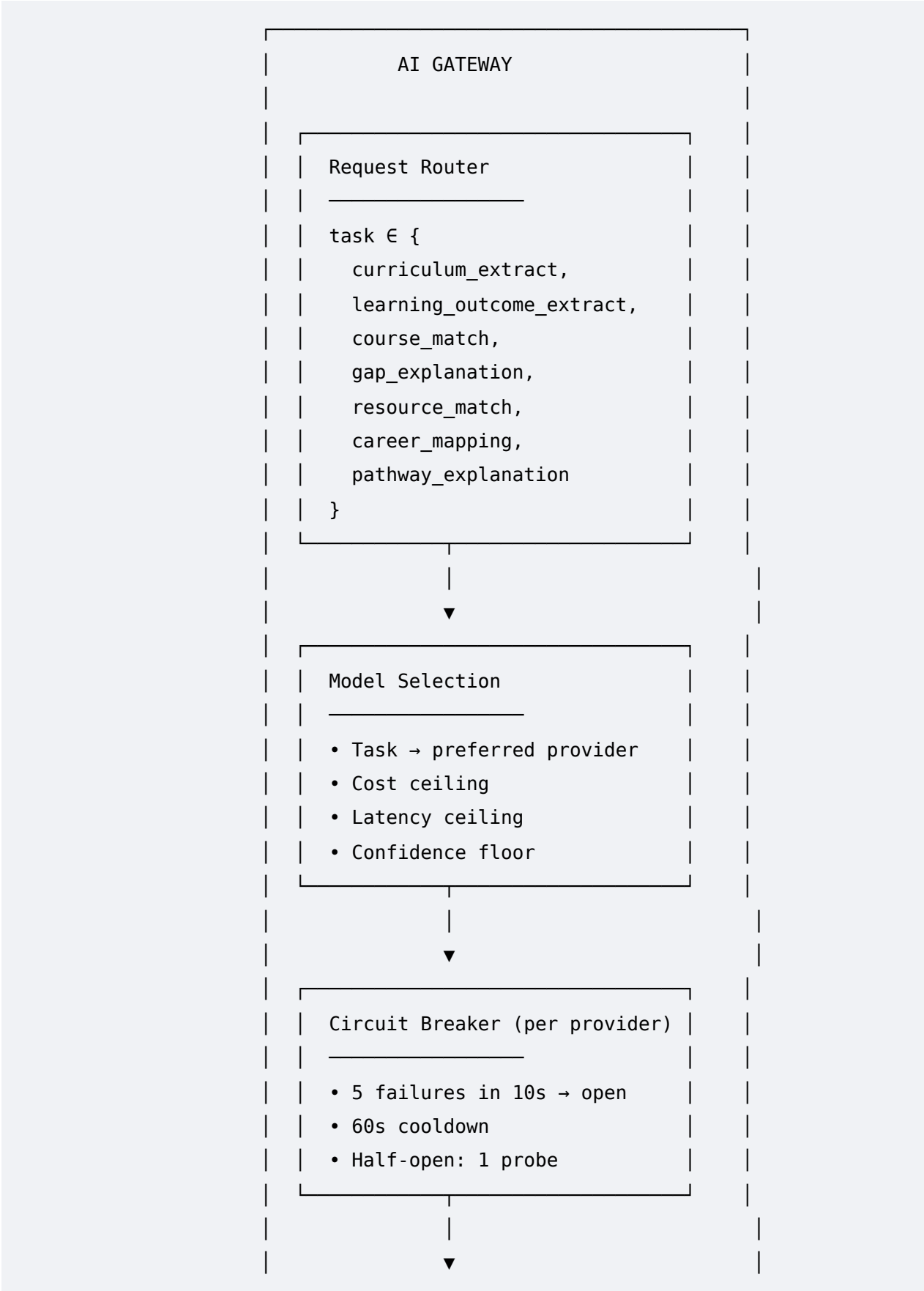
```
s3://edupathai-raw-docs/
├─ curricula/
│  └─ IIT-Bombay/BTech-CSE/2026.1/
│     └─ syllabus.pdf
│     └─ extracted.json
│     └─ metadata.json
│     └─ ...
├─ transcripts/
│  └─ {student_id}/{timestamp}.pdf
└─ certificates/
   └─ {student_id}/{timestamp}.pdf

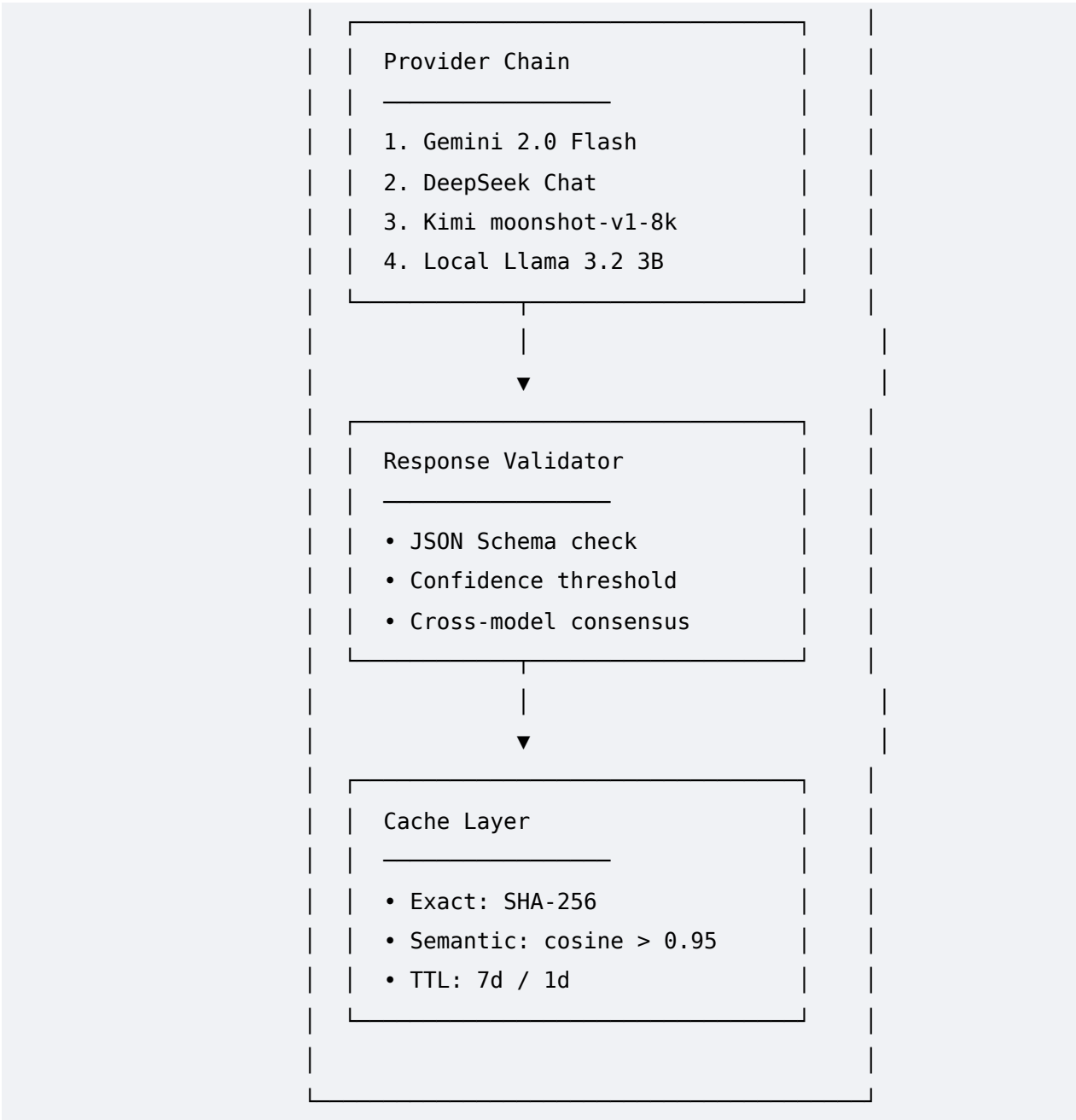
s3://edupathai-audit-archive/ [Object Lock: COMPLIANCE, 7 years]
├─ daily/
│  └─ 2026-09-11/
│     └─ chain-head.json
│     └─ records.jsonl
│     └─ signature.sig
└─ monthly/
   └─ 2026-09/
      └─ chain-head.json
      └─ signature.sig
```

6. AI/ML LAYER — DEEP DIVE

6.1 The AI Gateway

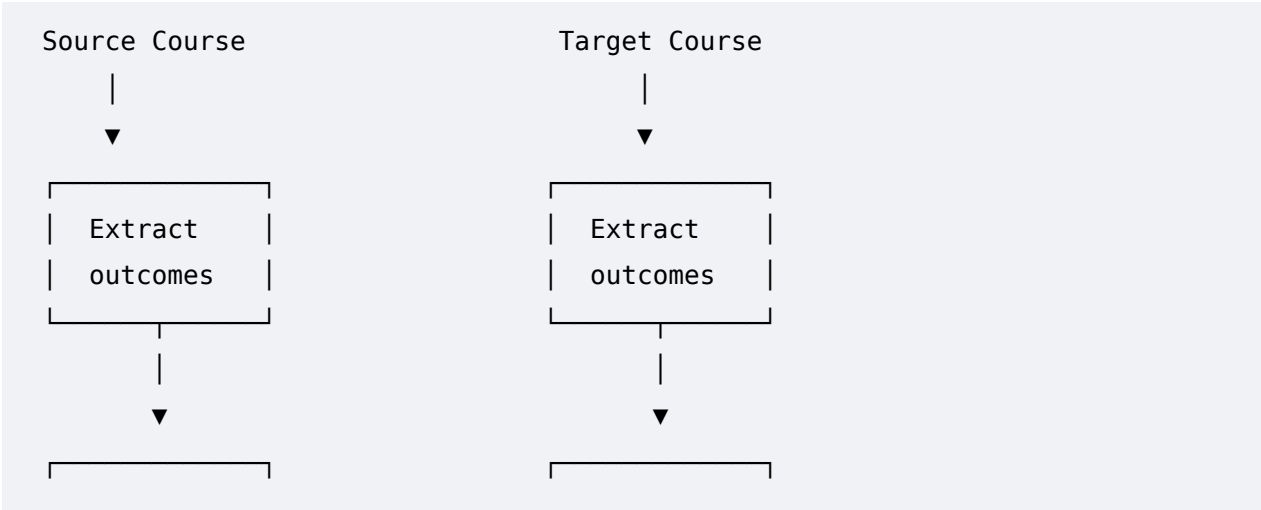
TEXT

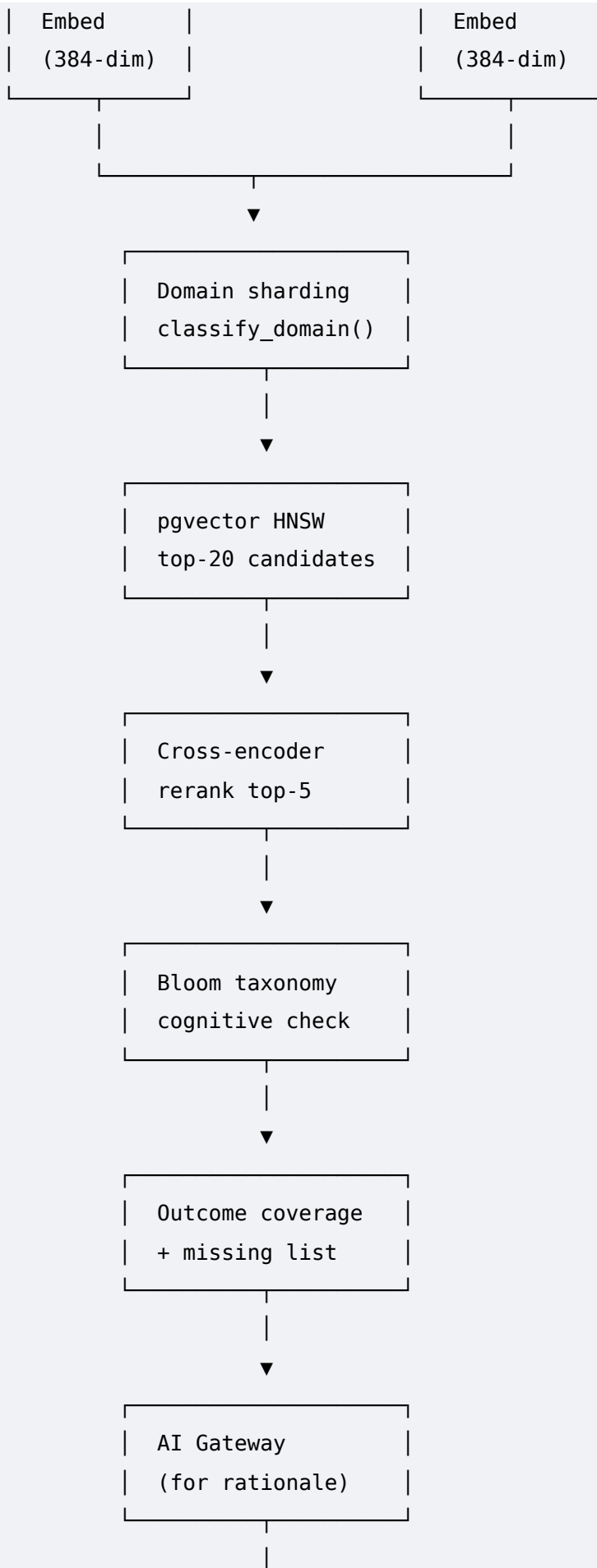




6.2 Matching Engine Pipeline

text





▼

```
| Decision consensus |  
| (agreement check) |
```

▼

```
MatchResult {  
    semantic_score,  
    outcome_coverage,  
    prerequisite_status,  
    assessment_match,  
    credit_compatibility,  
    domain_alignment,  
    policy_eligibility,  
    evidence_quality,  
    evidence: [...],  
    missing_outcomes: [...]  
}
```

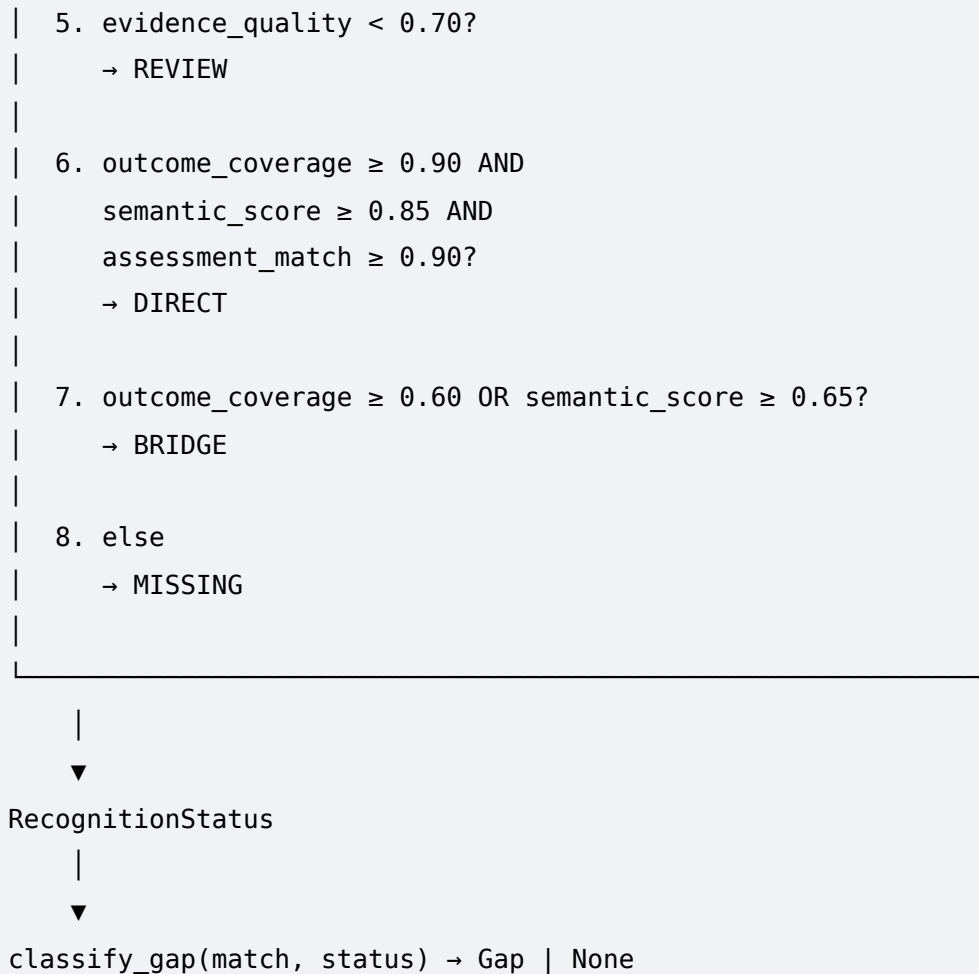
6.3 The Recognizer (Deterministic, No AI)

text

MatchResult

▼

```
|  
| classify(match, policy)  
|  
|  
| 1. policy_eligibility == False?  
|   → POLICY_CONFLICT  
|  
| 2. prerequisite_status == False?  
|   → MISSING  
|  
| 3. domain_alignment == False?  
|   → REVIEW  
|  
| 4. credit_compatibility == False?  
|   → REVIEW  
|
```



7. OPTIMIZATION LAYER — DEEP DIVE

7.1 MILP Formulation

text

Objective (weighted multi-objective):

minimize: $\lambda_1 \cdot \text{completion_time}$
 $+ \lambda_2 \cdot \text{bridge_burden}$
 $+ \lambda_3 \cdot \text{workload_variance}$
 $+ \lambda_4 \cdot \text{uncertainty}$
 $- \lambda_5 \cdot \text{preserved_learning}$

Decision variables:

$z[j, t] \in \{0,1\}$ course j taken in term t
 $u[b, t] \in \{0,1\}$ bridge b taken in term t
 $u_break[b, t] \in \{0,1\}$ bridge b in break before term t
 $w[t] \in \{0,1\}$ term t is active
 $\sigma[t] \geq 0$ workload deviation

Hard constraints:

- H1. $\sum_t z[j, t] = 1 \quad \forall j \in \text{remaining}$
- H2. $\sum_t u[b, t] = 1 \quad \forall b \in \text{formal_bridges}$
- H3. Prerequisite hyperedges satisfied
- H4. $\sum_j \text{credits}(j) \cdot z[j, t] + \text{bridge_hours}(t) \leq \text{max_credits} \cdot w[t]$
- H5. $\sum_j \text{credits}(j) \cdot z[j, t] \geq \text{min_credits} \cdot w[t] \quad (\text{except final term})$
- H6. $w[t] \leq w[t-1] \quad (\text{term monotonicity})$
- H7. $w[t] = 0 \text{ for } t > \text{max_terms}$
- H8. Residency: at least N terms at target institution

Modes:

- FASTEST: $\lambda = (3.0, 0.5, 0.2, 0.5, 0.5)$
- BALANCED: $\lambda = (1.0, 1.0, 1.0, 1.0, 1.0)$
- MAX_PRESERVATION: $\lambda = (0.5, 0.3, 0.5, 0.3, 3.0)$

7.2 Solver Fleet Sizing (Erlang-C)

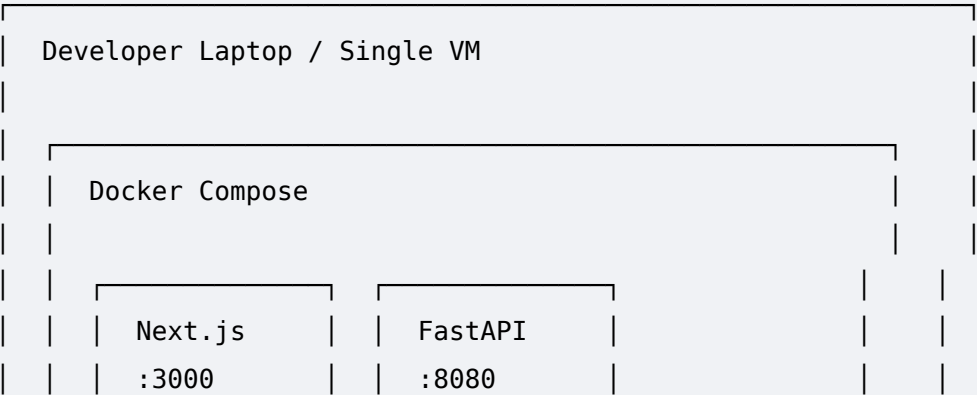
TEXT

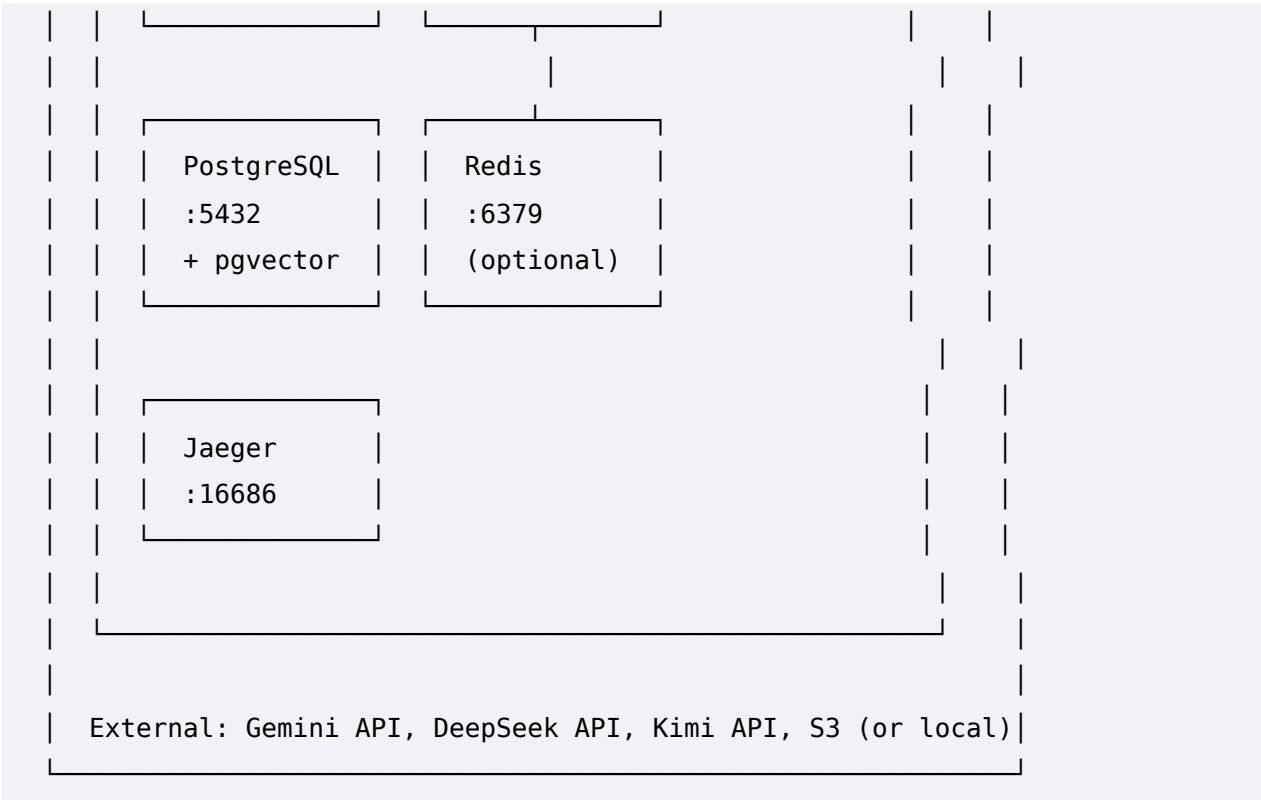
- Peak arrival rate: $\lambda = 8 \text{ req/s}$
- Average solve time: $E[S] = 2.0s$
- Service rate: $\mu = 0.5 \text{ req/s per worker}$
- Target: $P_{\text{wait}} < 0.10, p99 < 30s$
- Base: $c > \lambda/\mu = 16 \text{ workers}$
- Erlang-C: $c \approx 20 \text{ workers for } P_{\text{wait}} < 0.10$
- With cache (85% hit): $\lambda_{\text{eff}} = 1.2 \text{ req/s} \rightarrow c \approx 5 \text{ workers}$
- Burst protection: $c = 64 \text{ (4\times base)}$
- Queue threshold: $\text{depth} > 1000 \rightarrow \text{return REVIEW immediately}$

8. DEPLOYMENT TOPOLOGY (SIH → Production)

8.1 SIH Deployment (Single Machine)

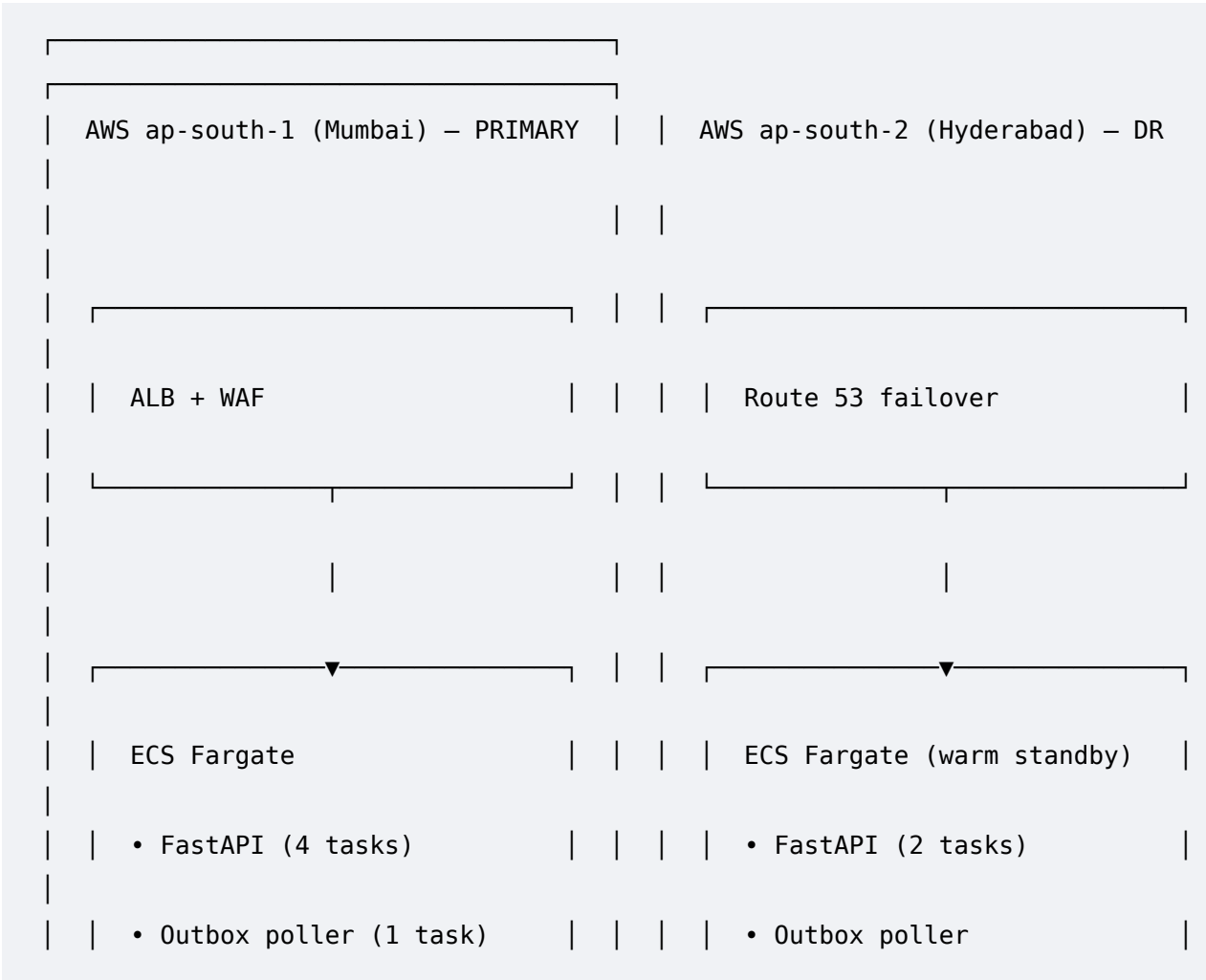
TEXT

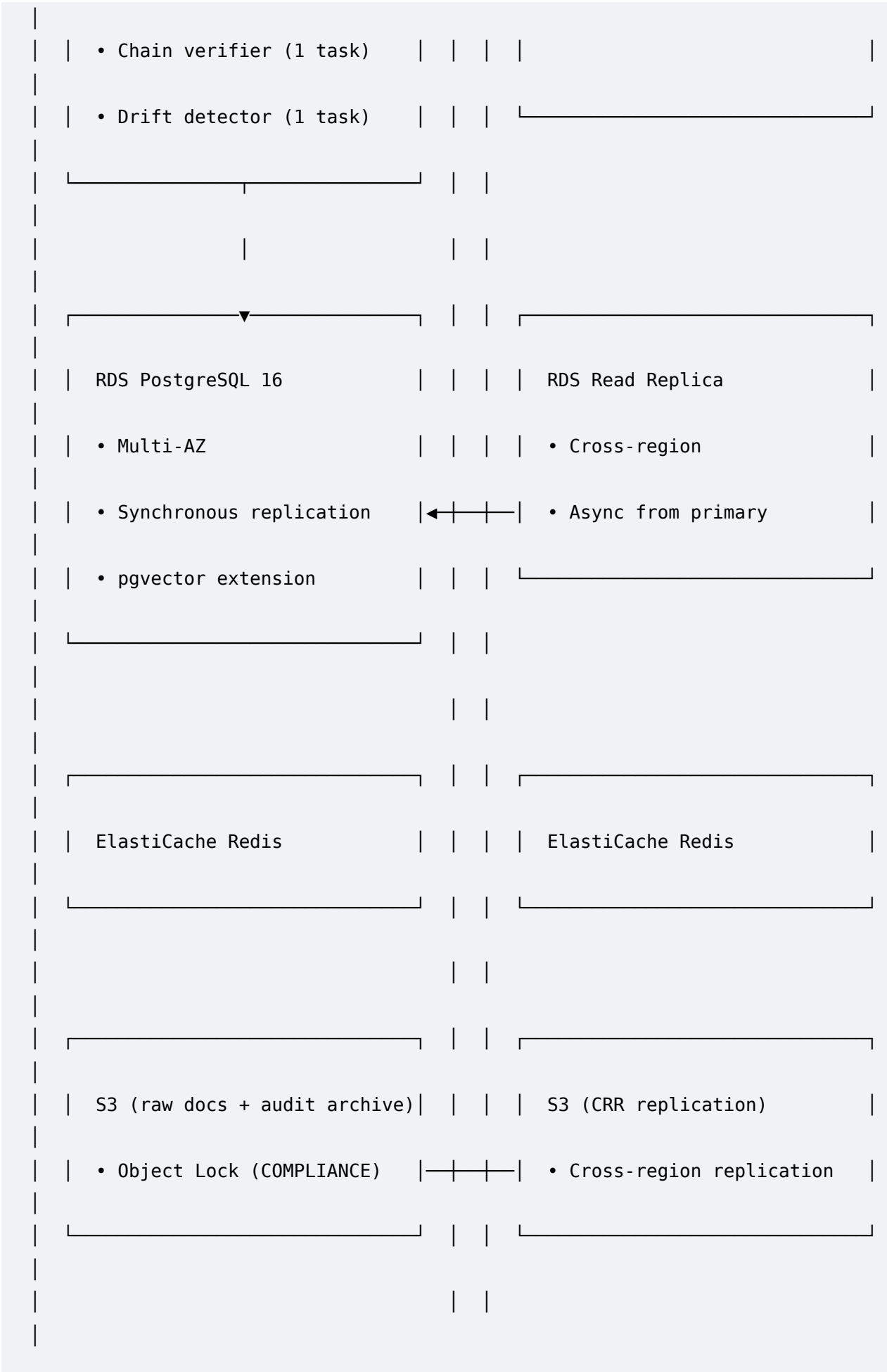


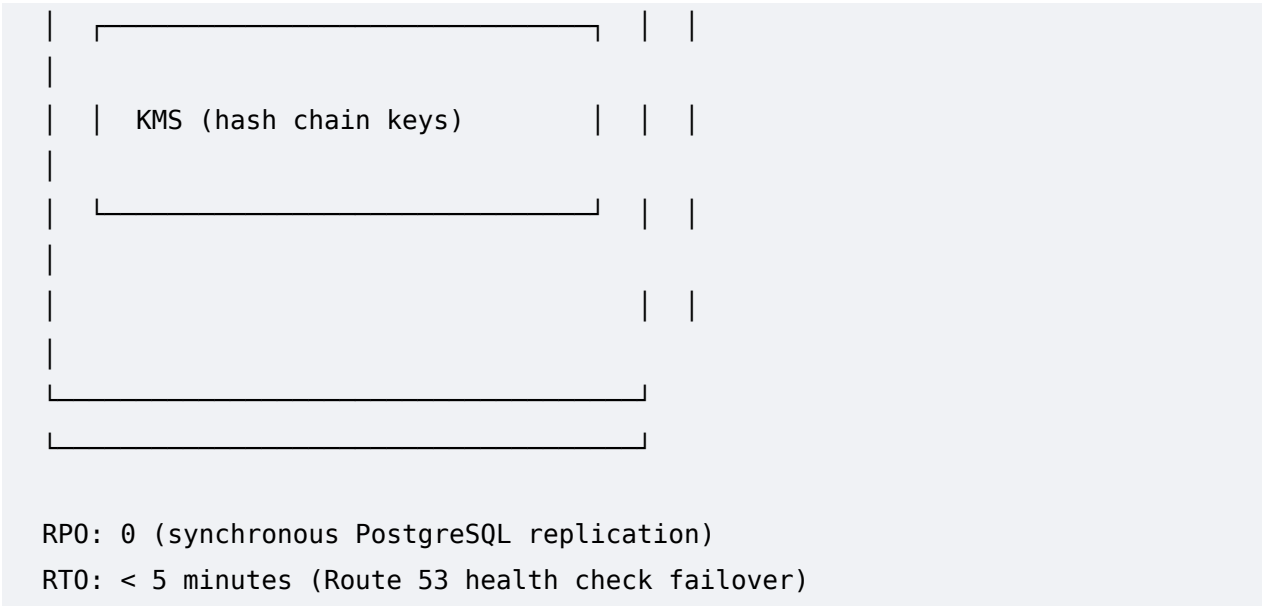


8.2 Production V1 (Single Region + DR)

text







9. OBSERVABILITY — WHAT WE MEASURE

9.1 Metric Families

Family	Metric	Target	Alert
System	API p95 latency	< 2s	> 5s
	MILP p95 solve time	< 15s	> 30s
	LLM call p95 latency	< 5s	> 15s
	PostgreSQL replication lag	< 100ms	> 1s
	Redis hit rate	> 80%	< 60%
	Error rate (5xx)	< 0.1%	> 1%

Pathway Correctness	False Equivalence Rate (FER)	< 2%	> 5%
	Recognition Precision	> 95%	< 90%
	Recognition Recall	> 90%	< 80%
	Constraint Violation Rate (CVR)	< 0.1%	> 1%
	Human Override Rate (HOR)	track/version	spike
	Infeasible Rate	track/version	spike
AI-Specific	Extraction confidence	> 0.85	< 0.70
	Cross- model consensus rate	> 80%	< 60%
	Human review rate	< 20%	> 40%
	Cache hit rate (LLM)	> 60%	< 40%

	Model drift (weekly)	< 5%	> 10%
	Cost per recommen dation	track	> 2× baseline
Circuit Breaker	Provider open state	0	> 0 for 5min
	Failure rate per provider	< 5%	> 10%
Audit	Chain integrity check	pass	any break
	Chain length per chain_id	growing	stalled
Solver Fleet	Queue depth	< 100	> 1000
	Worker utilization	60–80%	> 90%
	Cache hit rate	> 85%	< 60%

9.2 Distributed Tracing

text

```
Trace: pathway-request-abc123
├─ Span: api-gateway (5ms)
|   └─ attr: http.method=POST, http.route=/v1/pathway/request
├─ Span: orchestrator.run (2.4s)
|   └─ Span: snapshot.capture (12ms)
```

```

| | └─ attr: bundle_id=..., curriculum_version=...
| └─ Span: load-student-curriculum (15ms)
| └─ Span: matching.match (1.2s)
| | └─ Span: vector-search (200ms)
| | └─ Span: cross-encoder-rerank (800ms)
| | └─ Span: ai-gateway.call (3s)
| | | └─ Span: gemini.call (800ms) ✓
| | | └─ Span: deepseek.call (900ms) ✓
| | | └─ Span: consensus (5ms)
| | └─ Span: bloom-taxonomy-check (200ms)
| └─ Span: recognition.classify (2ms)
| └─ Span: gap.find (150ms)
| | └─ Span: hypergraph-traversal (100ms)
| | └─ Span: frontier-computation (50ms)
| └─ Span: bridge.recommend (10ms)
| └─ Span: solver.solve (8s)
| | └─ Span: cache.check (2ms) → miss
| | └─ Span: milp-formulation (500ms)
| | └─ Span: solver-execution (7s) → OPTIMAL
| | └─ Span: solution-extraction (500ms)
| └─ Span: audit.append (25ms)
| | └─ attr: chain_id=global, current_hash=abc...
| └─ Span: outbox.publish (5ms)
└─ Span: response-serialize (2ms)

```

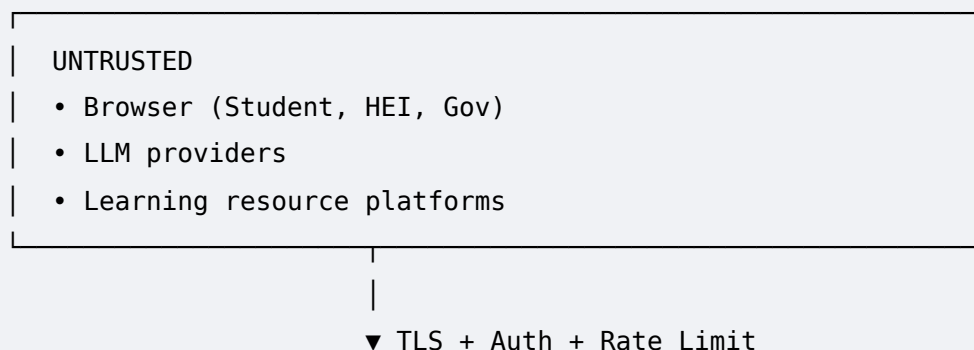
Key rule: `trace_id` is the OTel trace. `decision_id` is the business decision.

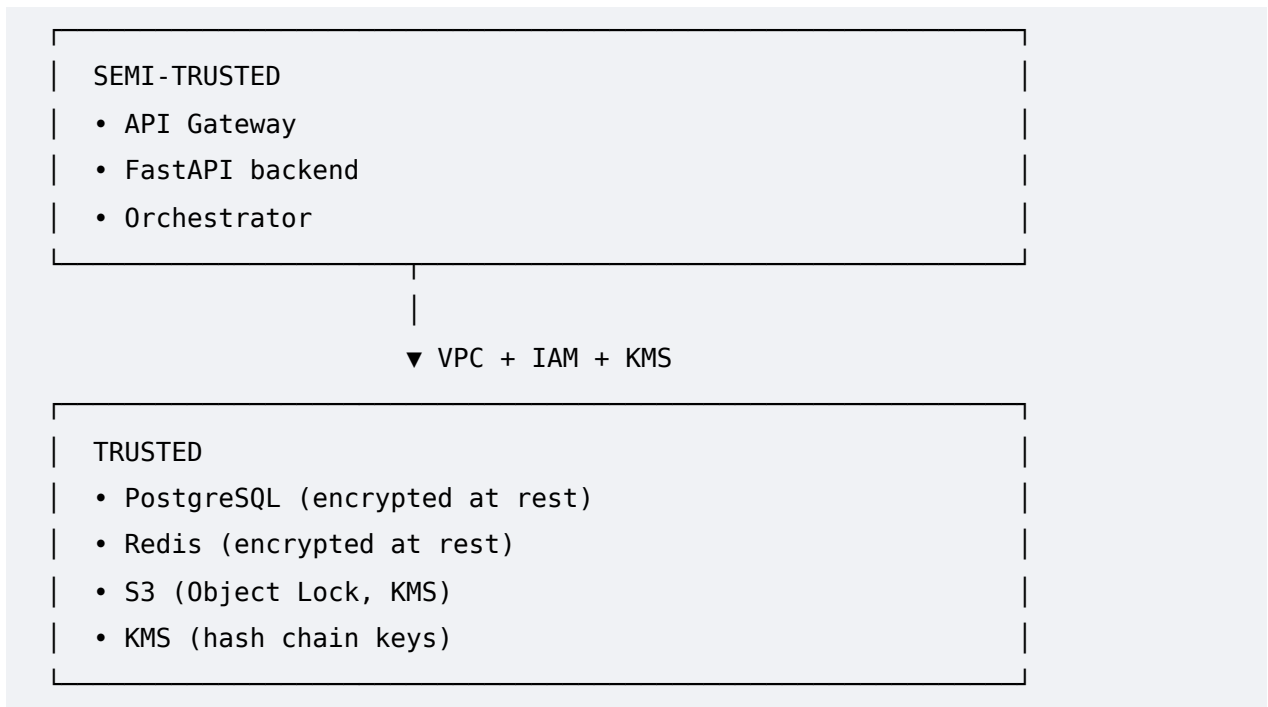
`audit_event_ids` are ledger entries. All three are linked but distinct. A student contesting a decision replays the exact same trace.

10. SECURITY & COMPLIANCE

10.1 Trust Boundaries

NOTE





10.2 DPDP Compliance Checklist

- ☐ Data minimization: no PII in `students` table (only tokenized ref)
- ☐ Consent: collected at registration, stored in consent log
- ☐ Purpose limitation: data used only for pathway recommendations
- ☐ Storage limitation: raw docs archived after 7 years
- ☐ Access control: RBAC (student / HEI / Gov / admin)
- ☐ Encryption: at rest (KMS) + in transit (TLS 1.3)
- ☐ Breach notification: alert on any chain break
- ☐ Right to erasure: student can request deletion of raw docs
- ☐ Audit trail: hash-chained, tamper-evident, immutable
- ☐ Cross-border: India-only (ap-south-1, ap-south-2)

11. TECHNOLOGY STACK SUMMARY

Layer	Technology	Version	Why
Frontend	Next.js	14.2.5	App Router, RSC, performance
	TypeScript	5.5.3	Type safety
	Tailwind CSS	3.4.7	Design tokens, responsive
	Framer Motion	11.3.19	Animations
	D3.js	7.9.0	Tree visualization
	React Query	5.51.1	Server state
	Zustand	4.5.4	Client state
Backend	Python	3.11.9	Team familiarity, ecosystem
	FastAPI	0.111.0	Async, Pydantic, OpenAPI

	Uvicorn	0.30.1	ASGI server
	Pydantic	2.7.4	Schema validation
	asyncpg	0.29.0	Async PostgreSQL
Database	PostgreSQL	16	ACID, recursive CTE, pgvector
	pgvector	0.7.0	Vector similarity
	Redis	7.2	Cache, sessions, locks
AI/ML	google-generativeai	0.7.2	Gemini SDK
	openai	1.35.0	DeepSeek, Kimi (OpenAI- compat)
	sentence-transformers	3.0.1	Embeddings, cross- encoder
	transformers	4.42.4	Local Llama fallback

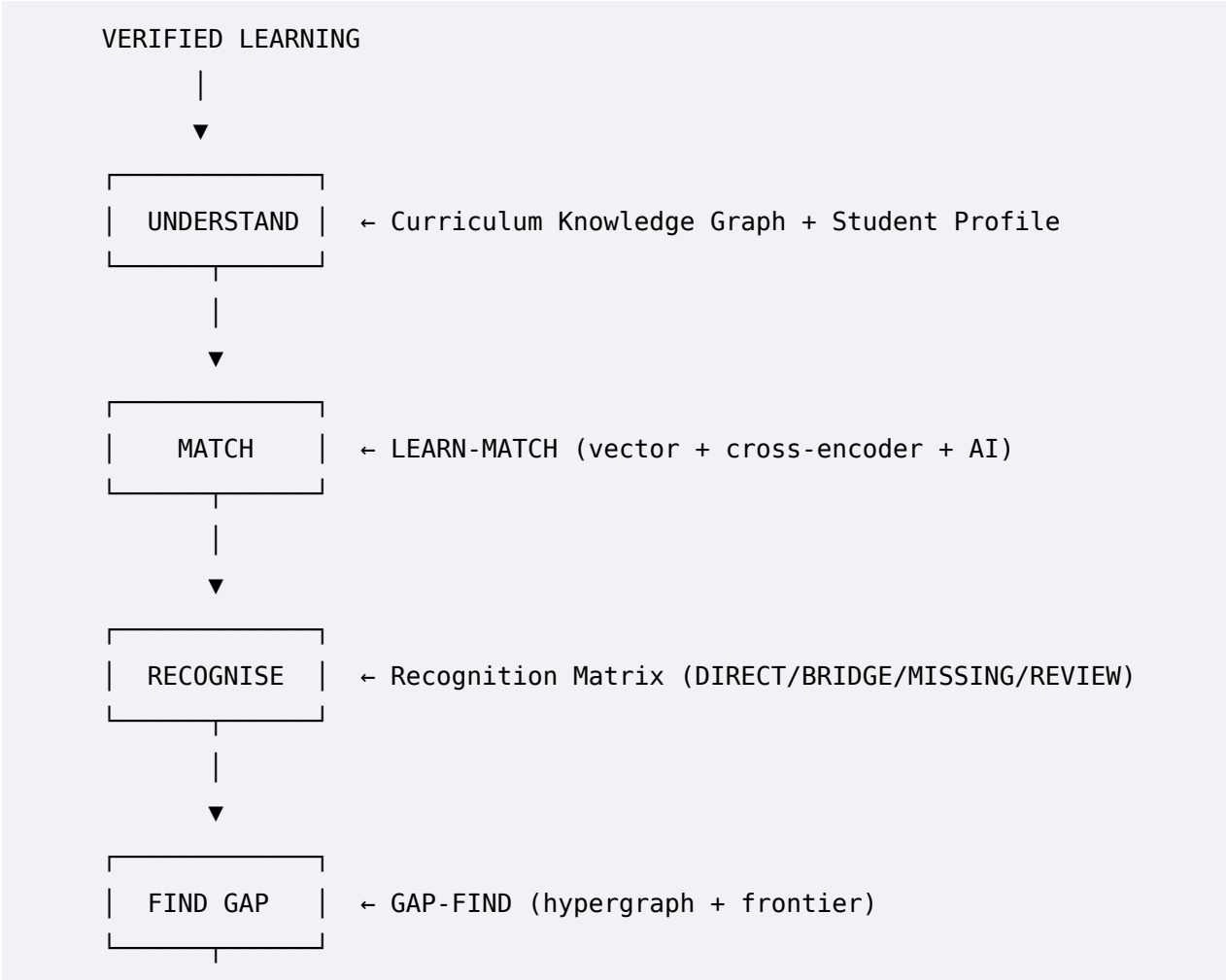
	torch	2.3.1	Model inference
Optimization	OR-Tools	9.10.4067	MILP solver (CBC)
	networkx	3.3	Graph algorithms
	numpy	1.26.4	Numeric
	scipy	1.14.0	Optimization utilities
Storage	S3	—	Object storage, Object Lock
	Git	—	Policy as code
Observability	OpenTelemetry	1.25.0	Traces, metrics, logs
	Prometheus	2.53	Metrics
	Grafana	11.1	Dashboards
	Jaeger	1.60	Distributed tracing
Infrastructure	Docker	26.1	Containerization

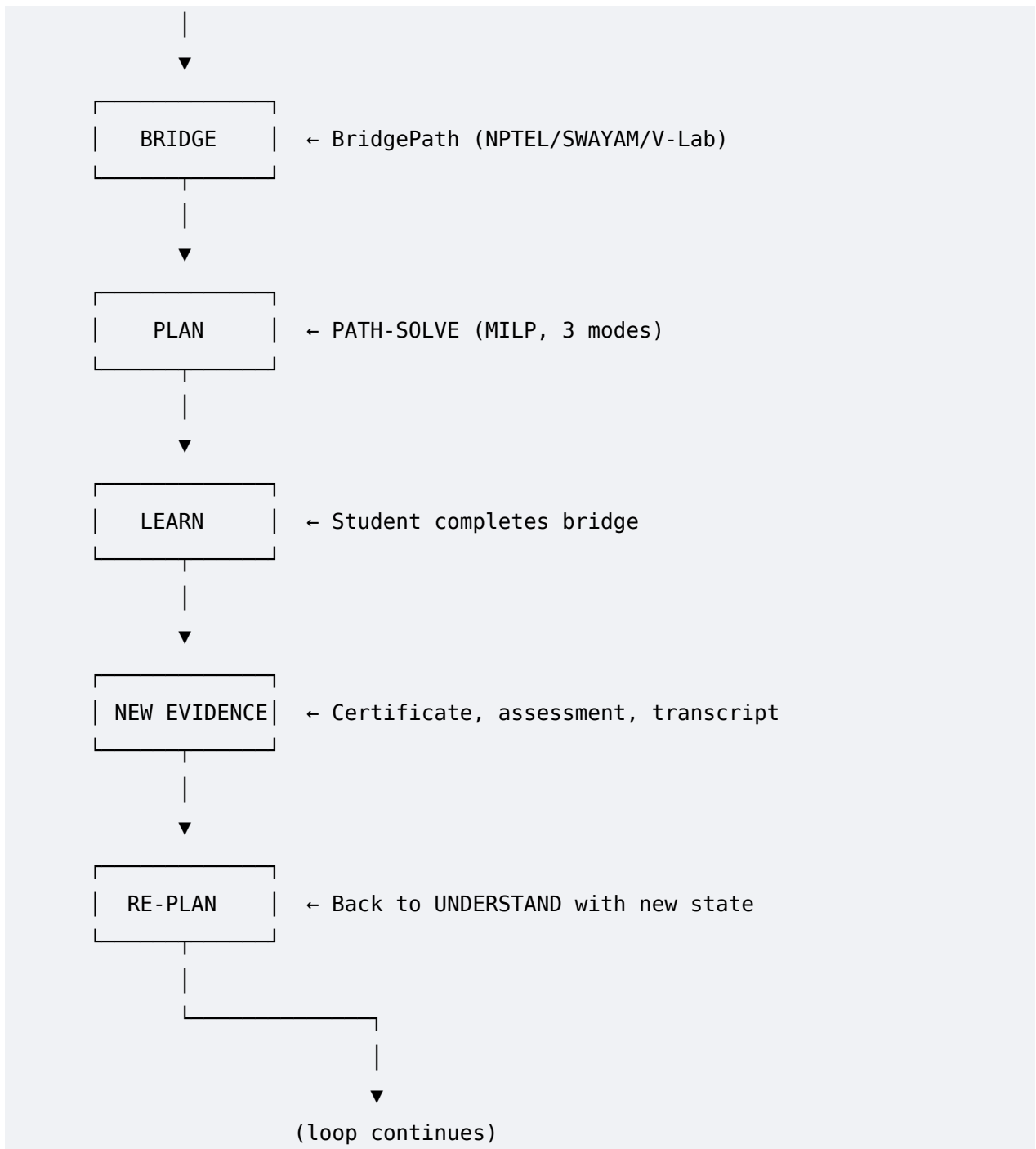
	Docker Compose	2.29	SIH deploymen t
	Terraform	1.8	Production IaC (V2)
	AWS ECS Fargate	—	Production compute (V2)
	AWS RDS	—	Production DB (V2)

12. THE DECISION LOOP (The Product)

This is what every component exists to serve:

TEXT





The tree is the visualization of this loop. The audit ledger is the memory. The AI Gateway is the interpretation. The MILP is the decision. The human is the gate.

13. SIH vs PRODUCTION — WHAT CHANGES

Component	SIH (36 hours)	Production V1
-----------	----------------	---------------

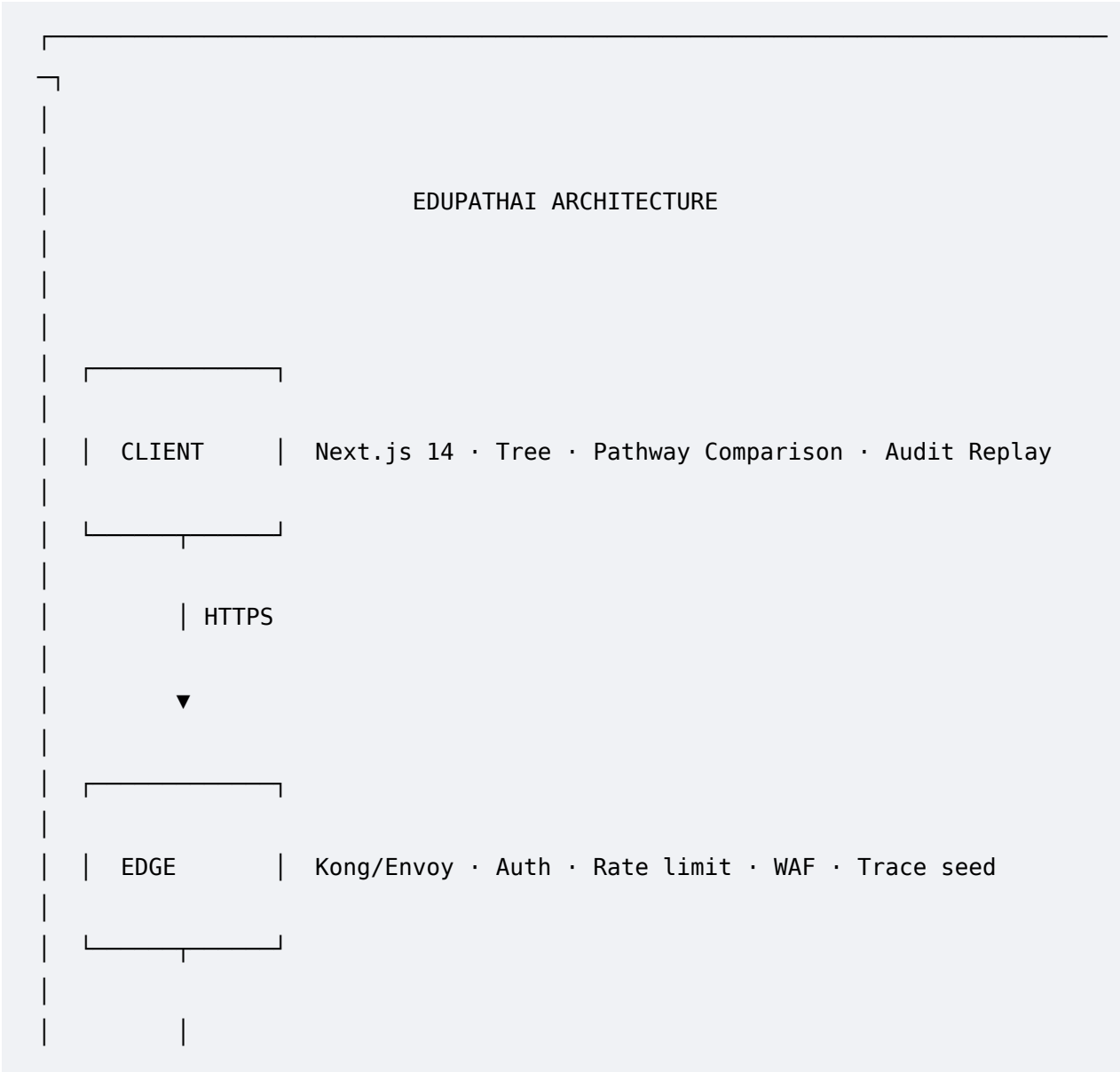
Frontend	Next.js dev server	Vercel / CloudFront + S3
Backend	Uvicorn single process	ECS Fargate (4 tasks)
Database	Docker Postgres	RDS Multi-AZ + read replica
Cache	Optional Redis	ElastiCache Redis cluster
Event bus	Postgres outbox + poller	Kafka (12 partitions, RF=3)
Analytics	Postgres materialized views	ClickHouse
AI Gateway	Direct API calls	Full gateway with cost tracking
Providers	Gemini + DeepSeek + Kimi + Local	Same + Claude + GPT-4
Circuit breaker	In-memory	In-memory + shared

		state via Redis
Consensus	2 providers	3+ providers
Solver	Single process	Fleet of 16–64 workers
Solver cache	Postgres table	Redis + Postgres
Audit chain	Single global chain	Partitioned (per institution)
KMS	Not used (SIH)	AWS KMS envelope encryption
S3 Object Lock	Not used	COMPLIANCE mode, 7 years
Multi-region	No	Active-passive (Mumbai + Hyderabad)
DR	No	RPO=0, RTO<5min
Observability	Jaeger local	OTel + Prometheus + Grafana

Security	Basic auth	OAuth2/OI DC + WAF + VPC
DPDP	Synthetic data	Full compliance
Hiring	6 students	12–16 professiona ls

14. THE ONE-PAGE SUMMARY

text





┌──────────┐

| APP | FastAPI · Orchestrator · Snapshot · Audit · Policy

| LAYER | Recognition · Gap · Frontier · Bridge · Outbox

└──────────┘

|



┌──────────┐

| AI/ML | AI Gateway · Matching · Consensus · Recognizer

| LAYER | pgvector · Cross-encoder · Bloom · Hypergraph

└──────────┘

|



┌──────────┐

| SOLVER | MILP · Fallback · Cache · Fleet

| LAYER | OR-Tools · 3 modes · Erlang-C sizing

└──────────┘

|



┌──────────┐



MEMBER 3 — FRONTEND / UX LEAD — THE ZERO-DAY ROADMAP (Python + Next.js)

"We don't render tables. We render the learner's academic future as a living, breathing tree — root is what they've learned, branches are what they can become. Every pixel earns its place. Every interaction teaches. Every page is a story."

0. WHAT MEMBER 3 OWNS (And What They Don't)

0.1 What Member 3 Owns

Deliverable	Owner	Consumer
Next.js app (all routes, all pages)	Member 3	Judge, Student, HEI, Gov
The Tree visualization (centerpiece)	Member 3	Judge's first "wow"
Design system (tokens, components)	Member 3	All pages
Responsive layouts (mobile/tablet/desktop)	Member 3	All users

Animations (Framer Motion)	Member 3	Delight factor
Accessibili ty (WCAG AA)	Member 3	All users
API integration (fetch from Member 1)	Member 3	All pages
Loading states, error boundaries	Member 3	Reliability
Dark mode + light mode	Member 3	User choice
Empty states, onboarding	Member 3	First impression
The 3- minute demo UI script	Member 3	Judge

0.2 What Member 3 Does NOT Own

Not Owned	Who Owns It
--------------	----------------

Backend API (endpoints, schemas)	Member 1
AI matching logic	Member 2
MILP solver	Member 4
Domain fixtures, curriculum data	Member 5
Pitch script, slide content	Member 6
Audit ledger internals	Member 1

0.3 The Critical Rule

Member 3 must NEVER invent API shapes. Everything comes from `services/schemas.py` (frozen by Member 1 at Hour 0). If the API doesn't provide something the UI needs, negotiate with Member 1 — do not mock it silently.

1. PRE-CODING: The "Zero Day" Setup (45 minutes)

1.1 Environment Lockdown

back

```
# 1. Verify repo structure
ls edupathai/services/schemas.py || echo "WAIT FOR MEMBER 1"
```

2. Create frontend structure

```
mkdir -p edupathai/web/{app,components,lib,hooks,styles,public}
mkdir -p edupathai/web/app/{student,hei,gov,about}
mkdir -p edupathai/web/components/{tree,layout,ui,charts,feedback}
mkdir -p edupathai/web/lib/{api,utils,constants}
mkdir -p edupathai/web/public/{fonts,icons,illustrations}
```

3. Node version

```
nvm install 20.14.0
nvm use 20.14.0
```

4. Bootstrap Next.js (App Router)

```
cd edupathai/web
npx create-next-app@14.2.5 . \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir=false \
  --import-alias="@/*"
```

5. PINNED dependencies (never `npm install` without version)

```
cat > package.json << 'EOF'
{
  "name": "edupathai-web",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "typecheck": "tsc --noEmit",
    "test": "vitest"
  },
  "dependencies": {
    "next": "14.2.5",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "typescript": "5.5.3",

    "@tanstack/react-query": "5.51.1",
```

```
"zustand": "4.5.4",
"zod": "3.23.8",

"tailwindcss": "3.4.7",
"tailwind-merge": "2.4.0",
"tailwindcss-animate": "1.0.7",
"class-variance-authority": "0.7.0",
"clsx": "2.1.1",

"@radix-ui/react-dialog": "1.1.1",
"@radix-ui/react-tabs": "1.1.0",
"@radix-ui/react-tooltip": "1.1.2",
"@radix-ui/react-popover": "1.1.1",
"@radix-ui/react-scroll-area": "1.1.0",
"@radix-ui/react-slot": "1.1.0",

"framer-motion": "11.3.19",
"d3-hierarchy": "3.1.2",
"d3-shape": "3.2.0",
"d3-zoom": "3.0.0",
"@types/d3-hierarchy": "3.1.7",
"@types/d3-shape": "3.1.6",
"@types/d3-zoom": "3.0.8",

"lucide-react": "0.417.0",
"recharts": "2.12.7",
"sonner": "1.5.0",

"next-themes": "0.3.0",
"date-fns": "3.6.0"
},
"devDependencies": {
  "@types/node": "20.14.12",
  "@types/react": "18.3.3",
  "@types/react-dom": "18.3.0",
  "eslint": "8.57.0",
  "eslint-config-next": "14.2.5",
  "postcss": "8.4.40",
  "autoprefixer": "10.4.19",
  "vitest": "2.0.5",
  "@testing-library/react": "16.0.0",
  "@testing-library/jest-dom": "6.4.8"
```

```
}  
}  
EOF
```

```
npm install
```

1.2 The Contract with Member 1 (Frozen API)

Create `web/lib/api/types.ts` — mirrors `services/schemas.py` exactly. Immutable.

typescript

```
// web/lib/api/types.ts  
// Mirror of services/schemas.py – frozen at Hour 0.  
// Any change requires re-negotiation with Member 1.  
  
export type RecognitionStatus =  
  | "DIRECT"  
  | "BRIDGE"  
  | "MISSING"  
  | "REVIEW"  
  | "POLICY_CONFLICT";  
  
export type GapType =  
  | "KNOWLEDGE"  
  | "PREREQUISITE"  
  | "ASSESSMENT"  
  | "ADMINISTRATIVE";  
  
export type PathwayMode = "FASTEST" | "BALANCED" | "MAX_PRESERVATION";  
  
export type SolverStatus = "OPTIMAL" | "FEASIBLE_NOT_OPTIMAL" | "HEURISTIC";  
  
export type BridgeMode = "LEARNING_ONLY" | "FORMAL_BRIDGE";  
  
export interface EvidenceRef {  
  source_doc_id: string;  
  source_page: string;  
  target_doc_id: string;  
  target_page: string;  
  similarity: number;  
}  
  
export interface MatchResult {  
  source_course_id: string;
```

```
target_course_id: string;
semantic_score: number;
outcome_coverage: number;
prerequisite_status: boolean;
assessment_match: number;
credit_compatibility: boolean;
domain_alignment: boolean;
policy_eligibility: boolean;
evidence_quality: number;
evidence: EvidenceRef[];
missing_outcomes: string[];
}

export interface Gap {
  gap_id: string;
  gap_type: GapType;
  description: string;
  missing_outcomes: string[];
}

export interface Bridge {
  bridge_id: string;
  gap_id: string;
  resource_id: string;
  resource_provider: string;
  resource_url: string;
  competency_coverage: number;
  duration_hours: number;
  assessment_available: boolean;
  recognition_status: BridgeMode;
  prerequisite_met: boolean;
}

export interface TermPlan {
  term_number: number;
  courses: string[];
  bridges: string[];
}

export interface Pathway {
  mode: PathwayMode;
  terms: number;
```

```
    bridge_burden: number;
    terms_plan: TermPlan[];
}

export interface DecisionBundle {
    id: string;
    curriculum_version: string;
    policy_version: string;
    model_version: string;
    prompt_version: string;
    embedding_model_version: string;
    cross_encoder_version: string;
    retrieval_threshold: number;
    solver_version: string;
    solver_parameters_hash: string;
    resource_catalog_version: string;
    ontology_version: string;
    ruleset_commit: string;
    tool_definitions_hash: string;
    captured_at: string;
}

export interface RecognitionSummary {
    direct: number;
    bridge: number;
    missing: number;
    review: number;
    policy_conflict: number;
}

export interface PathwayResponse {
    recognition: RecognitionSummary;
    gaps: Gap[];
    pathways: Pathway[];
    trace_id: string;
    decision_id: string;
    audit_event_ids: string[];
    bundle: DecisionBundle;
    // Extended by Member 1 for UI: matches + bridges
    matches?: MatchResult[];
    bridges?: Bridge[];
}
```

```

export interface AuditRecord {
  id: string;
  recommendation_id: string;
  decision_id: string;
  previous_hash: string;
  current_hash: string;
  chain_id: string;
  bundle_id: string;
  input_hash: string;
  output_hash: string;
  confidence: number;
  evidence: EvidenceRef[];
  ai_recommendation: string;
  human_decision: string | null;
  trace_id: string;
  timestamp: string;
}

```

1.3 The Tree Data Shape (UI-specific, derived from API)

Create `web/lib/tree/types.ts`:

typescript

```

// web/lib/tree/types.ts
// UI-specific transform of MatchResult[] → TreeLayout.

export type TreeNodeKind = "root" | "learned" | "unlockable" | "blocked" |
"gap";

export interface TreeNode {
  id: string;
  kind: TreeNodeKind;
  label: string;           // "CS-341: DBMS"
  status: RecognitionStatus | "ROOT";
  credits?: number;
  bloomLevel?: number;
  evidenceQuality?: number;
  // For "unlockable" nodes: what unlock this
  unlockedBy?: string[];
  // For gap nodes: what's missing
  missingOutcomes?: string[];
  // For root: total learned credits
}

```

```

    totalCredits?: number;
    // Children
    children?: TreeNode[];
  }

export interface TreeLayout {
  root: TreeNode;
  totalNodes: number;
  maxDepth: number;
  // Summary stats for header
  directCount: number;
  bridgeCount: number;
  missingCount: number;
  reviewCount: number;
}

```

1.4 The Design Tokens

Create `web/styles/tokens.css`:

CSS

```

/* web/styles/tokens.css
 * Design system tokens. Every color, space, radius, shadow lives here.
 * No hardcoded values in components.
 */

:root {
  /* === COLOR: Brand === */
  --brand-50: 250 245 255;
  --brand-100: 237 233 254;
  --brand-500: 139 92 246; /* violet-500 */
  --brand-600: 124 58 237; /* violet-600 */
  --brand-700: 109 40 217; /* violet-700 */

  /* === COLOR: Semantic (Recognition) === */
  --direct: 16 185 129; /* emerald-500 */
  --direct-bg: 236 253 245;
  --bridge: 245 158 11; /* amber-500 */
  --bridge-bg: 255 251 235;
  --missing: 244 63 94; /* rose-500 */
  --missing-bg: 255 241 242;
  --review: 100 116 139; /* slate-500 */
  --review-bg: 241 245 249;
}

```

```
--conflict: 217 70 239;      /* fuchsia-500 */
--conflict-bg: 253 244 255;

/* === COLOR: Surface === */
--bg-canvas: 250 250 250;
--bg-surface: 255 255 255;
--bg-elevated: 255 255 255;
--border-subtle: 226 232 240;
--border-strong: 203 213 225;
--text-primary: 15 23 42;
--text-secondary: 71 85 105;
--text-muted: 148 163 184;

/* === SPACING (4px base) === */
--space-1: 0.25rem;
--space-2: 0.5rem;
--space-3: 0.75rem;
--space-4: 1rem;
--space-6: 1.5rem;
--space-8: 2rem;
--space-12: 3rem;
--space-16: 4rem;

/* === RADIUS === */
--radius-sm: 0.375rem;
--radius-md: 0.5rem;
--radius-lg: 0.75rem;
--radius-xl: 1rem;
--radius-full: 9999px;

/* === SHADOW === */
--shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
--shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1);
--shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
--shadow-glow: 0 0 24px rgb(var(--brand-500) / 0.4);

/* === TYPOGRAPHY === */
--font-sans: "Inter", system-ui, sans-serif;
--font-mono: "JetBrains Mono", monospace;
--font-display: "Cal Sans", "Inter", sans-serif;

/* === MOTION === */
```

```

--ease-out: cubic-bezier(0.16, 1, 0.3, 1);
--ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
--duration-fast: 150ms;
--duration-base: 250ms;
--duration-slow: 500ms;
}

.dark {
  --bg-canvas: 10 14 26;
  --bg-surface: 17 24 39;
  --bg-elevated: 30 41 59;
  --border-subtle: 30 41 59;
  --border-strong: 51 65 85;
  --text-primary: 248 250 252;
  --text-secondary: 203 213 225;
  --text-muted: 100 116 139;
}

```

Create `web/tailwind.config.ts`:

typescript

```

// web/tailwind.config.ts
import type { Config } from "tailwindcss";

const config: Config = {
  darkMode: "class",
  content: ["/app/**/*.ts,tsx", "/components/**/*.ts,tsx"],
  theme: {
    extend: {
      colors: {
        brand: {
          50: "rgb(var(--brand-50) / <alpha-value>)",
          100: "rgb(var(--brand-100) / <alpha-value>)",
          500: "rgb(var(--brand-500) / <alpha-value>)",
          600: "rgb(var(--brand-600) / <alpha-value>)",
          700: "rgb(var(--brand-700) / <alpha-value>)",
        },
        direct: {
          DEFAULT: "rgb(var(--direct) / <alpha-value>)",
          bg: "rgb(var(--direct-bg) / <alpha-value>)",
        },
        bridge: {

```

```
    DEFAULT: "rgb(var(--bridge) / <alpha-value>)",
    bg: "rgb(var(--bridge-bg) / <alpha-value>)",
  },
  missing: {
    DEFAULT: "rgb(var(--missing) / <alpha-value>)",
    bg: "rgb(var(--missing-bg) / <alpha-value>)",
  },
  review: {
    DEFAULT: "rgb(var(--review) / <alpha-value>)",
    bg: "rgb(var(--review-bg) / <alpha-value>)",
  },
  conflict: {
    DEFAULT: "rgb(var(--conflict) / <alpha-value>)",
    bg: "rgb(var(--conflict-bg) / <alpha-value>)",
  },
  canvas: "rgb(var(--bg-canvas) / <alpha-value>)",
  surface: "rgb(var(--bg-surface) / <alpha-value>)",
  elevated: "rgb(var(--bg-elevated) / <alpha-value>)",
  subtle: "rgb(var(--border-subtle) / <alpha-value>)",
  strong: "rgb(var(--border-strong) / <alpha-value>)",
  primary: "rgb(var(--text-primary) / <alpha-value>)",
  secondary: "rgb(var(--text-secondary) / <alpha-value>)",
  muted: "rgb(var(--text-muted) / <alpha-value>)",
},
fontFamily: {
  sans: ["var(--font-sans)"],
  mono: ["var(--font-mono)"],
  display: ["var(--font-display)"],
},
boxShadow: {
  glow: "var(--shadow-glow)",
},
transitionTimingFunction: {
  out: "var(--ease-out)",
  spring: "var(--ease-spring)",
},
animation: {
  "fade-in": "fadeIn 0.3s var(--ease-out)",
  "slide-up": "slideUp 0.4s var(--ease-out)",
  "tree-grow": "treeGrow 0.8s var(--ease-spring)",
  "pulse-glow": "pulseGlow 2s ease-in-out infinite",
}
```

```

    },
    keyframes: {
      fadeIn: {
        "0%": { opacity: "0" },
        "100%": { opacity: "1" },
      },
      slideUp: {
        "0%": { opacity: "0", transform: "translateY(8px)" },
        "100%": { opacity: "1", transform: "translateY(0)" },
      },
      treeGrow: {
        "0%": { opacity: "0", transform: "scale(0.6)" },
        "100%": { opacity: "1", transform: "scale(1)" },
      },
      pulseGlow: {
        "0%, 100%": { boxShadow: "0 0 0 0 rgb(var(--brand-500) / 0.4)" },
        "50%": { boxShadow: "0 0 0 12px rgb(var(--brand-500) / 0)" },
      },
    },
  },
  plugins: [require("tailwindcss-animate")],
};

export default config;

```

2. PHASE 1: THE TREE (Hour 0–6) — The Centerpiece

Priority: The tree is the first thing a judge sees. It must be marvelous. It must work on mobile. It must tell a story.

2.1 The Tree Data Transform

Create `web/lib/tree/transform.ts`:

typescript

```

// web/lib/tree/transform.ts
// Transforms MatchResult[] + RecognitionSummary → TreeLayout.
// Root = student's learned courses. Children = unlockable / blocked.

import type {
  MatchResult, RecognitionSummary, Gap, PathwayResponse,
} from "@lib/api/types";
import type { TreeNode, TreeLayout, TreeNodeKind } from "../types";

```

```

export function buildTree(response: PathwayResponse): TreeLayout {
  const matches = response.matches ?? [];
  const gaps = response.gaps ?? [];

  // 1. Group by source course (root's children = learned courses)
  const sourceGroups = new Map<string, MatchResult[]>();
  for (const m of matches) {
    if (!sourceGroups.has(m.source_course_id)) {
      sourceGroups.set(m.source_course_id, []);
    }
    sourceGroups.get(m.source_course_id)!.push(m);
  }

  // 2. Build root
  const totalLearnedCredits = sourceGroups.size * 4; // approximation
  const root: TreeNode = {
    id: "root",
    kind: "root",
    label: "Your Academic Tree",
    status: "ROOT",
    totalCredits: totalLearnedCredits,
    children: [],
  };

  // 3. For each source course, build a learned node with children
  let maxDepth = 1;
  for (const [sourceCode, group] of sourceGroups.entries()) {
    const learnedNode: TreeNode = {
      id: `learned-${sourceCode}`,
      kind: "learned",
      label: sourceCode,
      status: "DIRECT",
      children: [],
    };

    // Children = target courses this source unlocks
    for (const m of group) {
      const kind = statusToKind(m);
      const child: TreeNode = {
        id: `target-${m.target_course_id}`,
        kind,
      };
    }
  }
}

```

```

        label: m.target_course_id,
        status: deriveStatus(m),
        bloomLevel: undefined,
        evidenceQuality: m.evidence_quality,
        missingOutcomes: m.missing_outcomes,
    };

    // If BRIDGE, add gap children
    if (kind === "gap" && m.missing_outcomes.length > 0) {
        child.children = m.missing_outcomes.map((outcome, i) => ({
            id: `gap-${m.target_course_id}-${i}`,
            kind: "gap" as TreeNodeKind,
            label: outcome.slice(0, 40) + (outcome.length > 40 ? "..." : ""),
            status: "MISSING" as const,
        }));
        maxDepth = Math.max(maxDepth, 3);
    }

    learnedNode.children!.push(child);
}

root.children!.push(learnedNode);
maxDepth = Math.max(maxDepth, 2);
}

return {
    root,
    totalNodes: countNodes(root),
    maxDepth,
    directCount: response.recognition.direct,
    bridgeCount: response.recognition.bridge,
    missingCount: response.recognition.missing,
    reviewCount: response.recognition.review,
};
}

function statusToKind(m: MatchResult): TreeNodeKind {
    const status = deriveStatus(m);
    switch (status) {
        case "DIRECT": return "learned";
        case "BRIDGE": return "gap";
        case "MISSING": return "blocked";
    }
}

```

```

    case "REVIEW": return "blocked";
    case "POLICY_CONFLICT": return "blocked";
  }
}

function deriveStatus(m: MatchResult): MatchResult["evidence_quality"]
extends never ? never : any {
  // Local status derivation for UI (mirrors recognizer logic, but
  simplified)
  if (!m.policy_eligibility) return "POLICY_CONFLICT";
  if (!m.prerequisite_status) return "MISSING";
  if (!m.domain_alignment || !m.credit_compatibility) return "REVIEW";
  if (m.outcome_coverage >= 0.9 && m.semantic_score >= 0.85 &&
m.assessment_match >= 0.9) {
    return "DIRECT";
  }
  if (m.outcome_coverage >= 0.6 || m.semantic_score >= 0.65) {
    return "BRIDGE";
  }
  return "MISSING";
}

function countNodes(node: TreeNode): number {
  let count = 1;
  if (node.children) {
    for (const c of node.children) count += countNodes(c);
  }
  return count;
}

```

2.2 The D3 Hierarchy Layout

Create `web/lib/tree/layout.ts`:

typescript

```

// web/lib/tree/layout.ts
// D3 hierarchy layout for the tree.
// Two modes: 'radial' (desktop) and 'vertical' (mobile/tablet).

import { hierarchy, tree as d3Tree, type HierarchyPointNode } from "d3-
hierarchy";
import type { TreeNode } from "../types";

```

```

export interface LayoutOptions {
  width: number;
  height: number;
  mode: "radial" | "vertical";
}

export interface LayoutResult {
  nodes: HierarchyPointNode<TreeNode>[];
  links: { source: HierarchyPointNode<TreeNode>; target:
HierarchyPointNode<TreeNode> }[];
  bounds: { minX: number; maxX: number; minY: number; maxY: number };
}

export function layoutTree(root: TreeNode, opts: LayoutOptions): LayoutResult
{
  const h = hierarchy(root);
  const nodeSize = opts.mode === "radial" ? [40, 180] : [80, 200];
  const treeLayout = d3Tree<TreeNode>()
    .nodeSize(nodeSize as [number, number])
    .separation((a, b) => (a.parent === b.parent ? 1 : 1.5));

  const laid = treeLayout(h);
  const nodes = laid.descendants();
  const links = laid.links();

  const xs = nodes.map((n) => n.x);
  const ys = nodes.map((n) => n.y);

  return {
    nodes,
    links,
    bounds: {
      minX: Math.min(...xs),
      maxX: Math.max(...xs),
      minY: Math.min(...ys),
      maxY: Math.max(...ys),
    },
  };
}

// Convert (x, y) from D3 to SVG coords based on mode
export function toSvgCoords(

```

```

    node: HierarchyPointNode<TreeNode>,
    opts: LayoutOptions,
  ): { x: number; y: number } {
    const [x, y] = [node.x, node.y];
    if (opts.mode === "radial") {
      // Radial: root at center, children outward
      const angle = (x / 180) * Math.PI;
      const radius = y;
      return {
        x: opts.width / 2 + radius * Math.cos(angle),
        y: opts.height / 2 + radius * Math.sin(angle),
      };
    }
    // Vertical: root at top, children below
    return {
      x: opts.width / 2 + x,
      y: 80 + y,
    };
  }
}

```

2.3 The Tree Component

Create `web/components/tree/Tree.tsx`:

tsx

```

"use client";

import { useMemo, useRef, useState, useEffect } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { useRouter } from "next/navigation";

import type { PathwayResponse } from "@lib/api/types";
import { buildTree } from "@lib/tree/transform";
import { layoutTree, toSvgCoords, type LayoutOptions } from
"@lib/tree/layout";
import type { TreeNode } from "@lib/tree/types";
import { TreeEdge } from "../TreeEdge";
import { TreeNode as TreeNodeComponent } from "../TreeNode";
import { TreeDetailPanel } from "../TreeDetailPanel";
import { TreeLegend } from "../TreeLegend";
import { useTreeLayoutMode } from "@hooks/useTreeLayoutMode";

interface TreeProps {

```

```

    response: PathwayResponse;
  }

export function Tree({ response }: TreeProps) {
  const router = useRouter();
  const containerRef = useRef<HTMLDivElement>(null);
  const [dimensions, setDimensions] = useState({ width: 1200, height: 800 });
  const [selectedNode, setSelectedNode] = useState<TreeNode | null>(null);
  const [hoveredNode, setHoveredNode] = useState<TreeNode | null>(null);

  // Responsive: switch to vertical on small screens
  const mode = useTreeLayoutMode(); // 'radial' | 'vertical'

  const treeData = useMemo(() => buildTree(response), [response]);

  const layoutOpts: LayoutOptions = useMemo(() => ({
    width: dimensions.width,
    height: dimensions.height,
    mode,
  }), [dimensions, mode]);

  const layout = useMemo(() => layoutTree(treeData.root, layoutOpts),
[treeData, layoutOpts]);

  // Resize observer
  useEffect(() => {
    if (!containerRef.current) return;
    const ro = new ResizeObserver((entries) => {
      const { width, height } = entries[0].contentRect;
      setDimensions({ width, height });
    });
    ro.observe(containerRef.current);
    return () => ro.disconnect();
  }, []);

  return (
    <div ref={containerRef} className="relative h-full w-full overflow-hidden
bg-canvas">
      {/* Header overlay */}
      <div className="pointer-events-none absolute left-6 top-6 z-10">
        <h1 className="font-display text-3xl font-semibold text-primary">
          Your Academic Tree

```

```

</h1>
<p className="mt-1 text-sm text-secondary">
  {treeData.directCount} direct · {treeData.bridgeCount} bridge · {"
"}
  {treeData.missingCount} missing
</p>
</div>

{/* Legend */}
<div className="absolute right-6 top-6 z-10">
  <TreeLegend />
</div>

{/* SVG Tree */}
<svg
  width={dimensions.width}
  height={dimensions.height}
  className="absolute inset-0"
  role="tree"
  aria-label="Academic pathway tree"
>
  <defs>
    <radialGradient id="rootGlow" cx="50%" cy="50%">
      <stop offset="0%" stopColor="rgb(var(--brand-500))"
stopOpacity="0.6" />
      <stop offset="100%" stopColor="rgb(var(--brand-500))"
stopOpacity="0" />
    </radialGradient>
  </defs>

  {/* Edges first (behind nodes) */}
  <g>
    {layout.links.map((link, i) => (
      <TreeEdge
        key={` ${link.source.data.id}-${link.target.data.id}`}
        source={toSvgCoords(link.source, layoutOpts)}
        target={toSvgCoords(link.target, layoutOpts)}
        status={link.target.data.status}
        isHighlighted={
          hoveredNode?.id === link.source.data.id ||
          hoveredNode?.id === link.target.data.id ||
          selectedNode?.id === link.source.data.id ||

```

```

        selectedNode?.id === link.target.data.id
      }
      delay={i * 0.02}
    />
  )})
</g>

{/* Nodes */}
<g>
  {layout.nodes.map((node, i) => {
    const pos = toSvgCoords(node, layoutOpts);
    return (
      <TreeNodeComponent
        key={node.data.id}
        node={node.data}
        x={pos.x}
        y={pos.y}
        delay={i * 0.03}
        onHover={setHoveredNode}
        onSelect={setSelectedNode}
        isSelected={selectedNode?.id === node.data.id}
      />
    );
  })}
</g>
</svg>

{/* Detail panel (slides in from right) */}
<AnimatePresence>
  {selectedNode && (
    <TreeDetailPanel
      node={selectedNode}
      response={response}
      onClose={() => setSelectedNode(null)}
      onViewPathway={(mode) => router.push(`/student/pathways?
mode=${mode}`)}
    />
  )}
</AnimatePresence>
</div>

```

```
);  
}
```

2.4 The Tree Node

Create `web/components/tree/TreeNode.tsx`:

tsx

```
"use client";  
  
import { motion } from "framer-motion";  
import type { TreeNode as TreeNodeType, TreeNodeKind } from  
"@/lib/tree/types";  
import { cn } from "@/lib/utils/cn";  
  
interface Props {  
  node: TreeNodeType;  
  x: number;  
  y: number;  
  delay: number;  
  onHover: (node: TreeNodeType | null) => void;  
  onSelect: (node: TreeNodeType) => void;  
  isSelected: boolean;  
}  
  
const STATUS_COLORS: Record<string, string> = {  
  ROOT: "fill-brand-500",  
  DIRECT: "fill-direct",  
  BRIDGE: "fill-bridge",  
  MISSING: "fill-missing",  
  REVIEW: "fill-review",  
  POLICY_CONFLICT: "fill-conflict",  
};  
  
const STATUS_RINGS: Record<string, string> = {  
  ROOT: "stroke-brand-700",  
  DIRECT: "stroke-direct",  
  BRIDGE: "stroke-bridge",  
  MISSING: "stroke-missing",  
  REVIEW: "stroke-review",  
  POLICY_CONFLICT: "stroke-conflict",  
};  
  
export function TreeNode({ node, x, y, delay, onHover, onSelect, isSelected
```

```

}): Props) {
  const radius = node.kind === "root" ? 36 : node.kind === "learned" ? 24 :
18;
  const fill = STATUS_COLORS[node.status] ?? "fill-review";
  const stroke = STATUS_RINGS[node.status] ?? "stroke-review";

  return (
    <motion.g
      initial={{ opacity: 0, scale: 0.6 }}
      animate={{ opacity: 1, scale: 1 }}
      transition={{ delay, duration: 0.5, ease: [0.34, 1.56, 0.64, 1] }}
      onMouseEnter={() => onHover(node)}
      onMouseLeave={() => onHover(null)}
      onClick={() => onSelect(node)}
      style={{ cursor: "pointer" }}
      role="treeitem"
      aria-label={node.label}
      aria-selected={isSelected}
      tabIndex={0}
      onKeyDown={(e) => {
        if (e.key === "Enter" || e.key === " ") {
          e.preventDefault();
          onSelect(node);
        }
      }}
    >
      { /* Glow for root */ }
      { node.kind === "root" && (
        <circle cx={x} cy={y} r={radius * 2} fill="url(#rootGlow)" />
      ) }

      { /* Selection ring */ }
      { isSelected && (
        <motion.circle
          cx={x}
          cy={y}
          r={radius + 6}
          fill="none"
          stroke="rgb(var(--brand-500))"
          strokeWidth={2}
          initial={{ scale: 0.8, opacity: 0 }}

```

```

        animate={{ scale: 1, opacity: 1 }}
        transition={{ duration: 0.2 }}
      />
    )}

    {/* Node circle */}
    <circle
      cx={x}
      cy={y}
      r={radius}
      className={cn(fill, stroke)}
      strokeWidth={2}
    />

    {/* Label */}
    <text
      x={x}
      y={y + radius + 16}
      textAnchor="middle"
      className="fill-primary text-[11px] font-medium select-none"
      style={{ pointerEvents: "none" }}
    >
      {truncate(node.label, 16)}
    </text>

    {/* Credits badge for learned nodes */}
    {node.kind === "learned" && node.credits && (
      <text
        x={x}
        y={y + 4}
        textAnchor="middle"
        className="fill-white text-[10px] font-semibold select-none"
        style={{ pointerEvents: "none" }}
      >
        {node.credits}
      </text>
    )}
  </motion.g>
);
}

```

```

function truncate(s: string, n: number): string {

```

```
    return s.length > n ? s.slice(0, n - 1) + "..." : s;
  }
```

2.5 The Tree Edge

Create `web/components/tree/TreeEdge.tsx`:

tsx

```
"use client";

import { motion } from "framer-motion";
import { linkHorizontal } from "d3-shape";

interface Props {
  source: { x: number; y: number };
  target: { x: number; y: number };
  status: string;
  isHighlighted: boolean;
  delay: number;
}

const STROKE_COLORS: Record<string, string> = {
  DIRECT: "stroke-direct",
  BRIDGE: "stroke-bridge",
  MISSING: "stroke-missing",
  REVIEW: "stroke-review",
  POLICY_CONFLICT: "stroke-conflict",
};

export function TreeEdge({ source, target, status, isHighlighted, delay }: Props) {
  const path = linkHorizontal<unknown, { x: number; y: number }>()
    .x((d) => d.x)
    .y((d) => d.y)({ source, target } as any);

  if (!path) return null;

  return (
    <motion.path
      d={path}
      fill="none"
      strokeWidth={isHighlighted ? 2.5 : 1.5}
      strokeDasharray={status === "MISSING" ? "4 4" : undefined}
      className={STROKE_COLORS[status] ?? "stroke-review"}
    />
  )
}
```

```

        initial={{ pathLength: 0, opacity: 0 }}
        animate={{ pathLength: 1, opacity: isHighlighted ? 1 : 0.5 }}
        transition={{ delay, duration: 0.6, ease: [0.16, 1, 0.3, 1] }}
      />
    );
  }
}

```

2.6 The Detail Panel

Create `web/components/tree/TreeDetailPanel.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import { X, BookOpen, AlertCircle, CheckCircle2, ArrowRight } from "lucide-react";

import type { TreeNode } from "@lib/tree/types";
import type { PathwayResponse } from "@lib/api/types";
import { cn } from "@lib/utils/cn";

interface Props {
  node: TreeNode;
  response: PathwayResponse;
  onClose: () => void;
  onViewPathway: (mode: string) => void;
}

export function TreeDetailPanel({ node, response, onClose, onViewPathway }: Props) {
  const statusConfig = {
    DIRECT: { icon: CheckCircle2, color: "text-direct", label: "Direct Recognition" },
    BRIDGE: { icon: AlertCircle, color: "text-bridge", label: "Bridge Required" },
    MISSING: { icon: AlertCircle, color: "text-missing", label: "Missing" },
    REVIEW: { icon: AlertCircle, color: "text-review", label: "Under Review" },
    POLICY_CONFLICT: { icon: AlertCircle, color: "text-conflict", label: "Policy Conflict" },
    ROOT: { icon: BookOpen, color: "text-brand-500", label: "Your Academic Tree" },
  };

```

```
}[node.status] ?? { icon: BookOpen, color: "text-muted", label: "Unknown"
};
```

```
const Icon = statusConfig.icon;
```

```
return (
  <motion.aside
    initial={{ x: 400, opacity: 0 }}
    animate={{ x: 0, opacity: 1 }}
    exit={{ x: 400, opacity: 0 }}
    transition={{ type: "spring", damping: 30, stiffness: 300 }}
    className="absolute right-0 top-0 z-20 h-full w-[400px] border-l
border-subtle bg-surface shadow-lg"
    role="complementary"
    aria-label={`Details for ${node.label}`}
  >
    <div className="flex h-full flex-col">
      {/* Header */}
      <div className="flex items-start justify-between border-b border-
subtle p-6">
        <div className="flex items-start gap-3">
          <Icon className={cn("h-6 w-6 shrink-0", statusConfig.color)} />
          <div>
            <h2 className="font-display text-lg font-semibold text-
primary">
              {node.label}
            </h2>
            <p className={cn("mt-1 text-sm font-medium",
statusConfig.color)}>
              {statusConfig.label}
            </p>
          </div>
        </div>
        <button
          onClick={onClose}
          className="rounded-md p-1.5 text-muted hover:bg-canvas
hover:text-primary"
          aria-label="Close details"
        >
          <X className="h-4 w-4" />
        </button>
      </div>
    </div>
  </div>
);
```

```

</div>

{/* Body */}
<div className="flex-1 overflow-y-auto p-6 space-y-6">
  {node.evidenceQuality !== undefined && (
    <Section title="Evidence Quality">
      <div className="flex items-center gap-2">
        <div className="h-2 flex-1 rounded-full bg-canvas">
          <motion.div
            className="h-2 rounded-full bg-brand-500"
            initial={{ width: 0 }}
            animate={{ width: `${node.evidenceQuality * 100}%` }}
            transition={{ duration: 0.6, ease: [0.16, 1, 0.3, 1] }}
          />
        </div>
        <span className="text-sm font-mono text-secondary">
          {(node.evidenceQuality * 100).toFixed(0)}%
        </span>
      </div>
    </Section>
  )}

  {node.missingOutcomes && node.missingOutcomes.length > 0 && (
    <Section title={`Missing Outcomes
    (${node.missingOutcomes.length})`} >
      <ul className="space-y-2">
        {node.missingOutcomes.map((outcome, i) => (
          <li key={i} className="flex items-start gap-2 text-sm text-
secondary">
            <span className="mt-1.5 h-1.5 w-1.5 shrink-0 rounded-full
bg-missing" />
            {outcome}
          </li>
        ))}
      </ul>
    </Section>
  )}

{/* Bridges for BRIDGE nodes */}
{node.status === "BRIDGE" && response.bridges && (
  <Section title="BridgePath Recommendations">
    <ul className="space-y-3">

```

```

        {response.bridges
          .filter((b) => node.missingOutcomes?.some((m) =>
            b.resource_id.toLowerCase().includes(m.slice(0,
6).toLowerCase()))
          .slice(0, 3)
          .map((bridge) => (
            <li key={bridge.bridge_id} className="rounded-lg border
border-subtle p-3">
              <p className="text-sm font-medium text-primary">
                {bridge.resource_provider}
              </p>
              <p className="mt-1 text-xs text-secondary">
                {bridge.duration_hours}h ·
{bridge.assessment_available ? "Assessment" : "Self-paced"}
              </p>
              <a
                href={bridge.resource_url}
                target="_blank"
                rel="noopener noreferrer"
                className="mt-2 inline-flex items-center gap-1 text-
xs font-medium text-brand-600 hover:text-brand-700"
              >
                View resource <ArrowRight className="h-3 w-3" />
              </a>
            </li>
          )})
        </ul>
      </Section>
    )}
  </div>

  {/* Footer actions */}
  <div className="border-t border-subtle p-6">
    <button
      onClick={() => onViewPathway("BALANCED")}
      className="w-full rounded-lg bg-brand-500 px-4 py-2.5 text-sm
font-medium text-white transition-colors hover:bg-brand-600"
    >
      View pathways for this course
    </button>
  </div>

```

```

        </div>
      </div>
    </motion.aside>
  );
}

function Section({ title, children }: { title: string; children:
React.ReactNode }) {
  return (
    <div>
      <h3 className="mb-3 text-xs font-semibold uppercase tracking-wider
text-muted">
        {title}
      </h3>
      {children}
    </div>
  );
}

```

2.7 The Legend

Create `web/components/tree/TreeLegend.tsx`:

tsx

```

"use client";

const ITEMS = [
  { label: "Direct", color: "bg-direct" },
  { label: "Bridge", color: "bg-bridge" },
  { label: "Missing", color: "bg-missing" },
  { label: "Review", color: "bg-review" },
];

export function TreeLegend() {
  return (
    <div className="rounded-lg border border-subtle bg-surface/80 p-3
backdrop-blur-sm">
      <ul className="space-y-1.5">
        {ITEMS.map((item) => (
          <li key={item.label} className="flex items-center gap-2 text-xs
text-secondary">
            <span className={`h-2.5 w-2.5 rounded-full ${item.color}`} />
            {item.label}
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```

        </li>
      )})
    </ul>
  </div>
);
}

```

2.8 The Responsive Hook

Create `web/hooks/useTreeLayoutMode.ts`:

typescript

```

"use client";

import { useState, useEffect } from "react";

export function useTreeLayoutMode(): "radial" | "vertical" {
  const [mode, setMode] = useState<"radial" | "vertical">("radial");

  useEffect(() => {
    const mq = window.matchMedia("(max-width: 1024px)");
    const handler = () => setMode(mq.matches ? "vertical" : "radial");
    handler();
    mq.addEventListener("change", handler);
    return () => mq.removeEventListener("change", handler);
  }, []);

  return mode;
}

```

Member 3's Hour 6 deliverable: The Tree renders. Nodes animate in from the root. Edges draw. Colors match recognition status. Detail panel slides in on click. Responsive switches radial ↔ vertical.

3. PHASE 2: THE APP SHELL + NAVIGATION (Hour 6–8)

3.1 Root Layout

Create `web/app/layout.tsx`:

tsx

```

import type { Metadata } from "next";
import { Inter, JetBrains_Mono } from "next/font/google";
import { ThemeProvider } from "next-themes";
import { QueryProvider } from "@components/providers/QueryProvider";

```

```

import { Toaster } from "sonner";
import "@styles/tokens.css";
import "@styles/globals.css";

const inter = Inter({ subsets: ["latin"], variable: "--font-sans" });
const jetbrains = JetBrains_Mono({ subsets: ["latin"], variable: "--font-mono" });

export const metadata: Metadata = {
  title: "EduPathAI – Academic Pathway Intelligence",
  description: "Stateful academic decision engine for NEP 2020",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" suppressHydrationWarning className={` ${inter.variable} ${jetbrains.variable}`}>
      <body className="bg-canvas font-sans text-primary antialiased">
        <ThemeProvider attribute="class" defaultTheme="system" enableSystem>
          <QueryProvider>
            {children}
          </QueryProvider>
          <Toaster position="bottom-right" />
        </ThemeProvider>
      </body>
    </html>
  );
}

```

3.2 The Navigation Shell

Create `web/components/layout/AppShell.tsx`:

`tsx`

```

"use client";

import { usePathname } from "next/navigation";
import Link from "next/link";
import { Home, TreePine, Route, ShieldCheck, Building2, BarChart3, Sun, Moon } from "lucide-react";
import { useTheme } from "next-themes";
import { cn } from "@lib/utils/cn";

```

```

const NAV = [
  { href: "/student", icon: Home, label: "Dashboard" },
  { href: "/student/tree", icon: TreePine, label: "Academic Tree" },
  { href: "/student/pathways", icon: Route, label: "Pathways" },
  { href: "/student/audit", icon: ShieldCheck, label: "Audit Trail" },
  { href: "/hei", icon: Building2, label: "HEI Review" },
  { href: "/gov", icon: BarChart3, label: "Mobility Intel" },
];

export function AppShell({ children }: { children: React.ReactNode }) {
  const pathname = usePathname();
  const { theme, setTheme } = useTheme();

  return (
    <div className="flex h-screen w-screen overflow-hidden bg-canvas">
      { /* Desktop sidebar */ }
      <nav className="hidden w-64 shrink-0 border-r border-subtle bg-surface
md:flex md:flex-col">
        <div className="flex h-16 items-center gap-2 border-b border-subtle
px-6">
          <div className="h-8 w-8 rounded-lg bg-gradient-to-br from-brand-500
to-brand-700" />
          <span className="font-display text-lg font-
semibold">EduPathAI</span>
        </div>

        <ul className="flex-1 space-y-1 p-3">
          {NAV.map((item) => {
            const isActive = pathname === item.href ||
pathname.startsWith(item.href + "/");
            const Icon = item.icon;
            return (
              <li key={item.href}>
                <Link
                  href={item.href}
                  className={cn(
                    "flex items-center gap-3 rounded-lg px-3 py-2 text-sm
font-medium transition-colors",
                    isActive
                      ? "bg-brand-50 text-brand-700 dark:bg-brand-500/10
dark:text-brand-500"
                      : "text-secondary hover:bg-canvas hover:text-primary"
                >

```

```

        })
      >
      <Icon className="h-4 w-4" />
      {item.label}
    </Link>
  </li>
);
}}
</ul>

<div className="border-t border-subtle p-3">
  <button
    onClick={() => setTheme(theme === "dark" ? "light" : "dark")}
    className="flex w-full items-center gap-3 rounded-lg px-3 py-2
text-sm font-medium text-secondary hover:bg-canvas"
  >
    {theme === "dark" ? <Sun className="h-4 w-4" /> : <Moon
className="h-4 w-4" />}
    Toggle theme
  </button>
</div>
</nav>

{/* Main content */}
<main className="flex-1 overflow-hidden">
  {children}
</main>

{/* Mobile bottom nav */}
<nav className="fixed bottom-0 left-0 right-0 z-30 border-t border-
subtle bg-surface md:hidden">
  <ul className="flex items-center justify-around">
    {NAV.slice(0, 5).map((item) => {
      const isActive = pathname === item.href;
      const Icon = item.icon;
      return (
        <li key={item.href}>
          <Link
            href={item.href}
            className={cn(
              "flex flex-col items-center gap-1 px-3 py-2 text-xs",
              isActive ? "text-brand-600" : "text-muted"
            )}
          >
            {Icon}
            {item.label}
          </Link>
        </li>
      )
    })}
  </ul>
</nav>

```

```

        })
      >
      <Icon className="h-5 w-5" />
      <span>{item.label.split(" ")[0]}</span>
    </Link>
  </li>
);
}}
</ul>
</nav>
</div>
);
}

```

3.3 The API Client

Create `web/lib/api/client.ts`:

typescript

```

// web/lib/api/client.ts
import type { PathwayResponse, AuditRecord } from "../types";

const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8080";

export async function requestPathway(
  studentId: string,
  targetProgramme: string,
  institution: string,
): Promise<PathwayResponse> {
  const res = await fetch(`${API_BASE}/v1/pathway/request`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ student_id: studentId, target_programme:
targetProgramme, institution }),
  });
  if (!res.ok) throw new Error(`Request failed: ${res.status}`);
  return res.json();
}

export async function getAudit(decisionId: string): Promise<AuditRecord> {
  const res = await fetch(`${API_BASE}/v1/audit/${decisionId}`);
  if (!res.ok) throw new Error(`Audit fetch failed: ${res.status}`);
}

```

```
    return res.json();
  }
```

3.4 The React Query Provider

Create `web/components/providers/QueryProvider.tsx`:

`tsx`

```
"use client";

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";

export function QueryProvider({ children }: { children: React.ReactNode }) {
  const [client] = useState(() => new QueryClient({
    defaultOptions: {
      queries: {
        staleTime: 60_000,
        refetchOnWindowFocus: false,
        retry: 1,
      },
    },
  }));
  return <QueryClientProvider client={client}>{children}</QueryClientProvider>;
}
```

Member 3's Hour 8 deliverable: App shell works. Sidebar navigation. Dark mode toggle. Mobile bottom nav. API client ready.

4. PHASE 3: THE STUDENT DASHBOARD (Hour 8–12)

4.1 The Dashboard Page

Create `web/app/student/page.tsx`:

`tsx`

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { motion } from "framer-motion";
import { Loader2, Sparkles } from "lucide-react";

import { requestPathway } from "@lib/api/client";
import { AppShell } from "@components/layout/AppShell";
```

```

import { StatCard } from "@components/charts/StatCard";
import { PathwayCard } from "@components/charts/PathwayCard";
import { GapList } from "@components/charts/GapList";
import { Tree } from "@components/tree/Tree";

const DEMO_STUDENT = "student-001";
const DEMO_PROGRAMME = "BTech-CSE";
const DEMO_INSTITUTION = "IIT-Bombay";

export default function StudentDashboard() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["pathway", DEMO_STUDENT, DEMO_PROGRAMME],
    queryFn: () => requestPathway(DEMO_STUDENT, DEMO_PROGRAMME,
DEMO_INSTITUTION),
  });

  if (isLoading) return <LoadingState />;
  if (error || !data) return <ErrorState />;

  return (
    <AppShell>
      <div className="h-full overflow-y-auto">
        </* Hero section with tree */>
        <section className="relative h-[60vh] min-h-[500px] border-b border-
subtle">
          <Tree response={data} />
        </section>

        </* Stats bar */>
        <section className="grid grid-cols-2 gap-4 p-6 md:grid-cols-4">
          <StatCard label="Direct Recognition" value=
{data.recognition.direct} tone="direct" delay={0} />
          <StatCard label="Bridge Required" value={data.recognition.bridge}
tone="bridge" delay={0.05} />
          <StatCard label="Missing" value={data.recognition.missing}
tone="missing" delay={0.1} />
          <StatCard label="Under Review" value={data.recognition.review}
tone="review" delay={0.15} />
        </section>

        </* Pathways */>
        <section className="p-6">

```

```

        <h2 className="font-display text-2xl font-semibold text-primary">
            Your Pathways
        </h2>
        <p className="mt-1 text-sm text-secondary">
            Three feasible routes to your target programme. Each preserves
different
            amounts of your prior learning.
        </p>

        <div className="mt-6 grid gap-6 md:grid-cols-3">
            {data.pathways.map((pathway, i) => (
                <PathwayCard key={pathway.mode} pathway={pathway} delay={i *
0.1} />
            ))}
        </div>
    </section>

    {/* Gaps */}
    {data.gaps.length > 0 && (
        <section className="p-6 pb-24">
            <h2 className="font-display text-2xl font-semibold text-primary">
                Learning Gaps
            </h2>
            <p className="mt-1 text-sm text-secondary">
                These competencies need bridging before your target programme.
            </p>
            <div className="mt-6">
                <GapList gaps={data.gaps} bridges={data.bridges ?? []} />
            </div>
        </section>
    )}
    </div>
</AppShell>
);
}

function LoadingState() {
    return (
        <AppShell>
            <div className="flex h-full items-center justify-center">
                <div className="flex flex-col items-center gap-3">
                    <Loader2 className="h-8 w-8 animate-spin text-brand-500" />

```

```

        <p className="text-sm text-secondary">Building your academic tree...
    </p>
    </div>
  </div>
</AppShell>
);
}

function ErrorState() {
  return (
    <AppShell>
      <div className="flex h-full items-center justify-center">
        <div className="text-center">
          <p className="text-lg font-medium text-primary">Unable to load your
pathway</p>
          <p className="mt-1 text-sm text-secondary">Please try again in a
moment.</p>
        </div>
      </div>
    </AppShell>
  );
}

```

4.2 Stat Card

Create `web/components/charts/StatCard.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import { cn } from "@/lib/utils/cn";

interface Props {
  label: string;
  value: number;
  tone: "direct" | "bridge" | "missing" | "review";
  delay?: number;
}

const TONES = {
  direct: { text: "text-direct", bg: "bg-direct-bg", border: "border-
direct/30" },

```

```

    bridge: { text: "text-bridge", bg: "bg-bridge-bg", border: "border-bridge/30" },
    missing: { text: "text-missing", bg: "bg-missing-bg", border: "border-missing/30" },
    review: { text: "text-review", bg: "bg-review-bg", border: "border-review/30" },
  };

export function StatCard({ label, value, tone, delay = 0 }: Props) {
  const t = TONES[tone];
  return (
    <motion.div
      initial={{ opacity: 0, y: 12 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ delay, duration: 0.4, ease: [0.16, 1, 0.3, 1] }}
      className={cn("rounded-xl border bg-surface p-5", t.border)}
    >
      <p className="text-xs font-medium uppercase tracking-wider text-muted">
{label}</p>
      <p className={cn("mt-2 font-display text-3xl font-semibold tabular-nums", t.text)}>
        {value}
      </p>
    </motion.div>
  );
}

```

4.3 Pathway Card

Create `web/components/charts/PathwayCard.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import { Zap, Scale, Shield } from "lucide-react";
import type { Pathway } from "@lib/api/types";
import { cn } from "@lib/utils/cn";

interface Props {
  pathway: Pathway;
  delay?: number;
}

```

```

const MODE_CONFIG = {
  FASTEST: { icon: Zap, label: "Fastest", description: "Shortest completion
time" },
  BALANCED: { icon: Scale, label: "Balanced", description: "Balanced
workload" },
  MAX_PRESERVATION: { icon: Shield, label: "Max Preservation", description:
"Most prior learning preserved" },
};

export function PathwayCard({ pathway, delay = 0 }: Props) {
  const config = MODE_CONFIG[pathway.mode];
  const Icon = config.icon;

  return (
    <motion.article
      initial={{ opacity: 0, y: 16 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ delay, duration: 0.4, ease: [0.16, 1, 0.3, 1] }}
      className="group relative overflow-hidden rounded-xl border border-
subtle bg-surface p-6 transition-shadow hover:shadow-md"
    >
      {/* Accent gradient */}
      <div className="absolute inset-x-0 top-0 h-1 bg-gradient-to-r from-
brand-500 to-brand-700" />

      <div className="flex items-start justify-between">
        <div>
          <div className="flex items-center gap-2">
            <Icon className="h-5 w-5 text-brand-500" />
            <h3 className="font-display text-lg font-semibold text-primary">
              {config.label}
            </h3>
          </div>
          <p className="mt-1 text-sm text-secondary">{config.description}</p>
        </div>
      </div>

      <dl className="mt-6 grid grid-cols-2 gap-4">
        <div>
          <dt className="text-xs font-medium uppercase tracking-wider text-
muted">Terms</dt>

```

```

      <dd className="mt-1 font-display text-2xl font-semibold tabular-
nums text-primary">
        {pathway.terms}
      </dd>
    </div>
    <div>
      <dt className="text-xs font-medium uppercase tracking-wider text-
muted">Bridge Load</dt>
      <dd className="mt-1 font-display text-2xl font-semibold tabular-
nums text-primary">
        {(pathway.bridge_burden * 100).toFixed(0)}%
      </dd>
    </div>
  </dl>

  /* Term plan visualization */
  <div className="mt-6 space-y-2">
    {pathway.terms_plan.slice(0, 5).map((term) => (
      <div key={term.term_number} className="flex items-center gap-2">
        <span className="w-12 shrink-0 text-xs font-mono text-muted">
          T{term.term_number}
        </span>
        <div className="flex-1 space-y-1">
          {term.courses.map((c) => (
            <div key={c} className="rounded bg-direct-bg px-2 py-1 text-
xs text-direct">
              {c}
            </div>
          ))}
          {term.bridges.map((b) => (
            <div key={b} className="rounded bg-bridge-bg px-2 py-1 text-
xs text-bridge">
              {b}
            </div>
          ))}
        </div>
      </div>
    ))}
  </div>

  <button className="mt-6 w-full rounded-lg border border-subtle px-4 py-

```

```

2 text-sm font-medium text-primary transition-colors hover:bg-canvas">
  View detailed plan
</button>
</motion.article>
);
}

```

4.4 Gap List

Create `web/components/charts/GapList.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import { BookOpen, AlertTriangle, ClipboardCheck, FileWarning } from "lucide-react";
import type { Gap, Bridge } from "@lib/api/types";
import { cn } from "@lib/utils/cn";

const GAP_ICONS = {
  KNOWLEDGE: BookOpen,
  PREREQUISITE: AlertTriangle,
  ASSESSMENT: ClipboardCheck,
  ADMINISTRATIVE: FileWarning,
};

const GAP_LABELS = {
  KNOWLEDGE: "Knowledge Gap",
  PREREQUISITE: "Prerequisite Gap",
  ASSESSMENT: "Assessment Gap",
  ADMINISTRATIVE: "Administrative Gap",
};

interface Props {
  gaps: Gap[];
  bridges: Bridge[];
}

export function GapList({ gaps, bridges }: Props) {
  return (
    <ul className="space-y-3">
      {gaps.map((gap, i) => {
        const Icon = GAP_ICONS[gap.gap_type];

```

```

const gapBridges = bridges.filter((b) => b.gap_id === gap.gap_id);
return (
  <motion.li
    key={gap.gap_id}
    initial={{ opacity: 0, x: -8 }}
    animate={{ opacity: 1, x: 0 }}
    transition={{ delay: i * 0.05, duration: 0.3 }}
    className="rounded-xl border border-subtle bg-surface p-5"
  >
    <div className="flex items-start gap-4">
      <div className="rounded-lg bg-bridge-bg p-2.5">
        <Icon className="h-5 w-5 text-bridge" />
      </div>
      <div className="flex-1">
        <h3 className="font-medium text-primary">
          {GAP_LABELS[gap.gap_type]}
        </h3>
        <p className="mt-1 text-sm text-secondary">{gap.description}</p>

        {gap.missing_outcomes.length > 0 && (
          <ul className="mt-3 space-y-1">
            {gap.missing_outcomes.map((outcome, j) => (
              <li key={j} className="flex items-start gap-2 text-xs text-secondary">
                <span className="mt-1.5 h-1 w-1 shrink-0 rounded-full bg-missing" />
                {outcome}
              </li>
            ))}
          </ul>
        )}

        {gapBridges.length > 0 && (
          <div className="mt-4 space-y-2">
            <p className="text-xs font-semibold uppercase tracking-wider text-muted">
              BridgePath Recommendations
            </p>
            {gapBridges.map((bridge) => (
              <a

```

```

        key={bridge.bridge_id}
        href={bridge.resource_url}
        target="_blank"
        rel="noopener noreferrer"
        className="block rounded-lg border border-subtle p-3
transition-colors hover:bg-canvas"
      >
        <div className="flex items-center justify-between">
          <span className="text-sm font-medium text-primary">
            {bridge.resource_provider}
          </span>
          <span className="text-xs text-muted">
            {bridge.duration_hours}h
          </span>
        </div>
        <div className="mt-2 flex items-center gap-2">
          <div className="h-1 flex-1 rounded-full bg-canvas">
            <div
              className="h-1 rounded-full bg-direct"
              style={{ width: `${bridge.competency_coverage *
100}%` }}
            />
          </div>
          <span className="text-xs text-secondary">
            {(bridge.competency_coverage * 100).toFixed(0)}%
coverage
          </span>
        </div>
        {bridge.recognition_status === "LEARNING_ONLY" && (
          <p className="mt-2 text-xs text-muted">
            ① Learning only – HEI approval required for
credit
          </p>
        )}
      </a>
    )})
  </div>
</div>
</div>
</motion.li>

```

```

    );
  })}
</ul>
);
}

```

Member 3's Hour 12 deliverable: Student dashboard complete. Tree is the hero. Stats, pathways, gaps all render. Animations work.

5. PHASE 4: PATHWAY VIEW + AUDIT REPLAY (Hour 12–18)

5.1 Pathway Comparison Page

Create `web/app/student/pathways/page.tsx`:

tsx

```

"use client";

import { useQuery } from "@tanstack/react-query";
import { useSearchParams } from "next/navigation";
import { requestPathway } from "@lib/api/client";
import { AppShell } from "@components/layout/AppShell";
import { PathwayComparison } from "@components/charts/PathwayComparison";
import { TermTimeline } from "@components/charts/TermTimeline";

export default function PathwaysPage() {
  const params = useSearchParams();
  const mode = params.get("mode");
  const { data, isLoading } = useQuery({
    queryKey: ["pathway"],
    queryFn: () => requestPathway("student-001", "BTech-CSE", "IIT-Bombay"),
  });

  if (isLoading || !data) return <div className="p-6">Loading...</div>;

  return (
    <AppShell>
      <div className="mx-auto max-w-6xl p-6 pb-24">
        <h1 className="font-display text-3xl font-semibold text-primary">
          Pathway Comparison
        </h1>
        <p className="mt-1 text-sm text-secondary">
          Three routes, side-by-side. Choose the one that fits your
          constraints.

```

```

    </p>

    <div className="mt-8">
      <PathwayComparison pathways={data.pathways} highlight={mode} />
    </div>

    <div className="mt-12">
      <h2 className="font-display text-2xl font-semibold text-primary">
        Term-by-Term Timeline
      </h2>
      <div className="mt-6">
        <TermTimeline pathways={data.pathways} />
      </div>
    </div>
  </AppShell>
);
}

```

5.2 Pathway Comparison Component

Create `web/components/charts/PathwayComparison.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import { Check } from "lucide-react";
import type { Pathway } from "@lib/api/types";
import { cn } from "@lib/utils/cn";

interface Props {
  pathways: Pathway[];
  highlight?: string | null;
}

export function PathwayComparison({ pathways, highlight }: Props) {
  return (
    <div className="grid gap-4 md:grid-cols-3">
      {pathways.map((pathway, i) => {
        const isHighlighted = highlight === pathway.mode;
        return (
          <motion.div
            key={pathway.mode}

```

```

        initial={{ opacity: 0, y: 12 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: i * 0.1 }}
        className={cn(
          "rounded-xl border bg-surface p-6",
          isHighlighted
            ? "border-brand-500 shadow-glow"
            : "border-subtle"
        )}
      >
        <h3 className="font-display text-lg font-semibold text-primary">
          {pathway.mode.replace("_", " ")}
        </h3>

        <dl className="mt-4 space-y-3">
          <Row label="Total Terms" value={pathway.terms} />
          <Row label="Bridge Burden" value={`$${(pathway.bridge_burden *
100).toFixed(0)}%`} />
          <Row label="Courses per Term" value=
{avgCoursesPerTerm(pathway)} />
        </dl>

        {isHighlighted && (
          <div className="mt-4 flex items-center gap-2 rounded-lg bg-
brand-50 p-3 text-sm text-brand-700 dark:bg-brand-500/10">
            <Check className="h-4 w-4" />
            Currently selected
          </div>
        )}
      </motion.div>
    );
  }}}
</div>
);
}

function Row({ label, value }: { label: string; value: string | number }) {
  return (
    <div className="flex items-center justify-between">
      <dt className="text-sm text-secondary">{label}</dt>
      <dd className="font-mono text-sm font-medium text-primary tabular-
nums">{value}</dd>

```

```

    </div>
  );
}

function avgCoursesPerTerm(p: Pathway): number {
  const total = p.terms_plan.reduce((sum, t) => sum + t.courses.length, 0);
  return Math.round((total / p.terms) * 10) / 10;
}

```

5.3 Term Timeline

Create `web/components/charts/TermTimeline.tsx`:

tsx

```

"use client";

import { motion } from "framer-motion";
import type { Pathway } from "@lib/api/types";

interface Props {
  pathways: Pathway[];
}

export function TermTimeline({ pathways }: Props) {
  const maxTerms = Math.max(...pathways.map((p) => p.terms));

  return (
    <div className="space-y-8">
      {pathways.map((pathway, i) => (
        <div key={pathway.mode}>
          <h3 className="mb-3 text-sm font-semibold uppercase tracking-wider text-muted">
            {pathway.mode.replace("_", " ")}
          </h3>
          <div className="flex gap-2 overflow-x-auto pb-2">
            {Array.from({ length: maxTerms }).map((_, termIdx) => {
              const term = pathway.terms_plan.find((t) => t.term_number ===
termIdx + 1);
              const isActive = !!term;
              return (
                <motion.div
                  key={termIdx}
                  initial={{ opacity: 0, scale: 0.9 }}
                  animate={{ opacity: 1, scale: 1 }}

```

```

        transition={{ delay: i * 0.05 + termIdx * 0.03 }}
        className={`min-w-[180px] rounded-lg border p-3 ${
          isActive ? "border-subtle bg-surface" : "border-dashed
border-subtle bg-canvas/50"
        }}
      >
      <p className="text-xs font-medium text-muted">Term {termIdx
+ 1}</p>

      {isActive ? (
        <ul className="mt-2 space-y-1">
          {term.courses.map((c) => (
            <li key={c} className="rounded bg-direct-bg px-2 py-1
text-xs text-direct">
              {c}
            </li>
          ))}
          {term.bridges.map((b) => (
            <li key={b} className="rounded bg-bridge-bg px-2 py-1
text-xs text-bridge">
              {b}
            </li>
          ))}
        </ul>
      ) : (
        <p className="mt-2 text-xs text-muted">Not required</p>
      )}
    </motion.div>
  );
  }}}
</div>
</div>
  )}
</div>
);
}

```

5.4 Audit Replay Page

Create `web/app/student/audit/[decisionId]/page.tsx`:

158

```
"use client";
```

```
import { useQuery } from "@tanstack/react-query";
import { motion } from "framer-motion";
import { FileText, Hash, Cpu, ScrollText, CheckCircle2, XCircle } from
"lucide-react";
import { getAudit } from "@lib/api/client";
import { AppShell } from "@components/layout/AppShell";

export default function AuditPage({ params }: { params: { decisionId: string
} }) {
  const { data, isLoading } = useQuery({
    queryKey: ["audit", params.decisionId],
    queryFn: () => getAudit(params.decisionId),
  });

  if (isLoading || !data) return <div className="p-6">Loading audit...</div>;

  return (
    <AppShell>
      <div className="mx-auto max-w-4xl p-6 pb-24">
        <div className="flex items-start justify-between">
          <div>
            <h1 className="font-display text-3xl font-semibold text-primary">
              Audit Trail
            </h1>
            <p className="mt-1 text-sm text-secondary">
              Every decision is reproducible. This is the evidence chain.
            </p>
          </div>
          <div className="rounded-lg border border-subtle bg-surface px-3 py-
2 font-mono text-xs text-secondary">
            {data.decision_id.slice(0, 8)}...
          </div>
        </div>

        </* Hash chain visualization */>
        <motion.section
          initial={{ opacity: 0, y: 12 }}
          animate={{ opacity: 1, y: 0 }}
          className="mt-8 rounded-xl border border-subtle bg-surface p-6"
        >
          <h2 className="flex items-center gap-2 font-display text-lg font-
semibold text-primary">
```

```

        <Hash className="h-5 w-5" />
        Hash Chain
    </h2>

    <div className="mt-4 space-y-2 font-mono text-xs">
        <HashRow label="Previous" value={data.previous_hash} />
        <div className="flex items-center gap-2 text-muted">
            <div className="h-3 w-px bg-subtle" />
            <span>↓ sha256</span>
        </div>
        <HashRow label="Current" value={data.current_hash} highlight />
    </div>
</motion.section>

{ /* Snapshot */}
<motion.section
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ delay: 0.1 }}
    className="mt-6 rounded-xl border border-subtle bg-surface p-6"
>
    <h2 className="flex items-center gap-2 font-display text-lg font-
semibold text-primary">
        <Cpu className="h-5 w-5" />
        Decision Bundle
    </h2>
    <dl className="mt-4 grid gap-3 sm:grid-cols-2">
        <BundleRow label="Curriculum" value={data.bundle_id} />
        <BundleRow label="Trace" value={data.trace_id} />
    </dl>
</motion.section>

{ /* AI vs Human */}
<motion.section
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ delay: 0.2 }}
    className="mt-6 grid gap-4 md:grid-cols-2"
>
    <div className="rounded-xl border border-subtle bg-surface p-6">
        <h3 className="text-sm font-semibold uppercase tracking-wider
text-muted">

```

```

        AI Recommendation
    </h3>
    <p className="mt-2 font-display text-2xl font-semibold text-
primary">
        {data.ai_recommendation}
    </p>
    <p className="mt-1 text-sm text-secondary">
        Confidence: {(data.confidence * 100).toFixed(1)}%
    </p>
</div>

<div className="rounded-xl border border-subtle bg-surface p-6">
    <h3 className="text-sm font-semibold uppercase tracking-wider
text-muted">
        Human Decision
    </h3>
    <p className="mt-2 font-display text-2xl font-semibold text-
primary">
        {data.human_decision ?? "-"}
    </p>
    {data.human_decision ? (
        <CheckCircle2 className="mt-2 h-5 w-5 text-direct" />
    ) : (
        <p className="mt-1 text-sm text-secondary">Awaiting BoS
review</p>
    )}
</div>
</motion.section>

{/* Evidence */}
<motion.section
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    transition={{ delay: 0.3 }}
    className="mt-6 rounded-xl border border-subtle bg-surface p-6"
>
    <h2 className="flex items-center gap-2 font-display text-lg font-
semibold text-primary">
        <FileText className="h-5 w-5" />
        Evidence ({data.evidence.length})
    </h2>

```

```

    <ul className="mt-4 space-y-3">
      {data.evidence.map((e, i) => (
        <li key={i} className="rounded-lg border border-subtle p-3">
          <div className="flex items-center justify-between text-sm">
            <span className="font-medium text-primary">
              {e.source_doc_id} → {e.target_doc_id}
            </span>
            <span className="font-mono text-xs text-muted">
              {(e.similarity * 100).toFixed(1)}%
            </span>
          </div>
          <p className="mt-1 text-xs text-secondary">
            Page {e.source_page} → Page {e.target_page}
          </p>
        </li>
      ))}
    </ul>
  </motion.section>
</div>
</AppShell>
);
}

```

```

function HashRow({ label, value, highlight }: { label: string; value: string;
highlight?: boolean }) {
  return (
    <div className={`rounded-lg border p-3 ${highlight ? "border-brand-500
bg-brand-50/50" : "border-subtle"}`>
      <span className="text-muted">{label}</span>{" "}
      <span className="break-all text-primary">{value}</span>
    </div>
  );
}

```

```

function BundleRow({ label, value }: { label: string; value: string }) {
  return (
    <div>
      <dt className="text-xs font-medium uppercase tracking-wider text-
muted">{label}</dt>
      <dd className="mt-1 break-all font-mono text-xs text-primary">{value}
    </dd>
  )
}

```

```
    </div>
  );
}
```

Member 3's Hour 18 deliverable: Pathway comparison, term timeline, audit replay all working.
Judge can click "why" on any decision and see the full evidence chain.

6. PHASE 5: HEI + GOVERNMENT VIEWS (Hour 18–22)

6.1 HEI Review Queue

Create `web/app/hei/page.tsx`:

tsx

```
"use client";

import { AppShell } from "@components/layout/AppShell";
import { ReviewQueue } from "@components/hei/ReviewQueue";

export default function HEIPage() {
  return (
    <AppShell>
      <div className="mx-auto max-w-6xl p-6 pb-24">
        <h1 className="font-display text-3xl font-semibold text-primary">
          BoS Review Queue
        </h1>
        <p className="mt-1 text-sm text-secondary">
          AI proposes. You decide. Every decision is auditable.
        </p>
        <div className="mt-8">
          <ReviewQueue />
        </div>
      </div>
    </AppShell>
  );
}
```

6.2 Government Dashboard

Create `web/app/gov/page.tsx`:

tsx

```
"use client";

import { AppShell } from "@components/layout/AppShell";
```

```
import { MobilityIntelligence } from "@components/gov/MobilityIntelligence";

export default function GovPage() {
  return (
    <AppShell>
      <div className="mx-auto max-w-7xl p-6 pb-24">
        <h1 className="font-display text-3xl font-semibold text-primary">
          Mobility Intelligence
        </h1>
        <p className="mt-1 text-sm text-secondary">
          Aggregate friction patterns across institutions. Privacy-
preserving.
        </p>
        <div className="mt-8">
          <MobilityIntelligence />
        </div>
      </div>
    </AppShell>
  );
}
```

Member 3's Hour 22 deliverable: HEI + Gov views render with mock data. Judge sees the full stakeholder story.

7. PHASE 6: RESPONSIVE POLISH + MOBILE (Hour 22–26)

7.1 Mobile-First Checklist

- ☐ Tree switches to vertical on mobile
- ☐ Tree becomes scrollable list on very small screens
- ☐ Detail panel becomes bottom sheet on mobile
- ☐ Stat cards stack 2×2 on mobile, 1×4 on desktop
- ☐ Pathway cards stack vertically on mobile
- ☐ Bottom nav on mobile (already in AppShell)

- ☐

Touch targets $\geq 44\text{px}$

- ☐

No horizontal scroll anywhere

- ☐

Font sizes scale appropriately

7.2 Mobile Tree Fallback

Create `web/components/tree/TreeMobile.tsx`:

tsx

```
"use client";

import { motion } from "framer-motion";
import { ChevronRight } from "lucide-react";
import type { TreeNode } from "@lib/tree/types";

interface Props {
  root: TreeNode;
  onSelect: (node: TreeNode) => void;
}

export function TreeMobile({ root, onSelect }: Props) {
  return (
    <div className="space-y-2 p-4">
      <div className="rounded-xl bg-gradient-to-br from-brand-500 to-brand-700 p-4 text-white">
        <p className="text-xs uppercase tracking-wider opacity-80">Root</p>
        <p className="mt-1 font-display text-lg font-semibold">{root.label}</p>
        <p className="mt-1 text-sm opacity-90">{root.totalCredits} credits learned</p>
      </div>

      <div className="space-y-1.5">
        {root.children?.map((child) => (
          <motion.button
            key={child.id}
            initial={{ opacity: 0, x: -8 }}
            animate={{ opacity: 1, x: 0 }}
            onClick={() => onSelect(child)}
          >
            {child.label}
            <ChevronRight />
          </motion.button>
        ))}
      </div>
    </div>
  );
}
```

```

        className="flex w-full items-center justify-between rounded-lg
border border-subtle bg-surface p-3 text-left"
      >
        <div>
          <p className="text-sm font-medium text-primary">{child.label}
</p>
          <p className="text-xs text-muted">{child.children?.length ?? 0}
branches</p>
        </div>
        <ChevronRight className="h-4 w-4 text-muted" />
      </motion.button>
    )}}
  </div>
</div>
);
}

```

Update `Tree.tsx`:

tsx

```

{isMobile ? (
  <TreeMobile root={treeData.root} onSelect={setSelectedNode} />
) : (
  <svg>...</svg>
)}

```

Add `useIsMobile` hook:

typescript

```

// web/hooks/useIsMobile.ts
"use client";
import { useState, useEffect } from "react";

export function useIsMobile(): boolean {
  const [isMobile, setIsMobile] = useState(false);
  useEffect(() => {
    const mq = window.matchMedia("(max-width: 640px)");
    const handler = () => setIsMobile(mq.matches);
    handler();
    mq.addEventListener("change", handler);
    return () => mq.removeEventListener("change", handler);
  }, []);
}

```

```
    return isMobile;
  }
```

Member 3's Hour 26 deliverable: Every page works on iPhone SE. No horizontal scroll. Touch targets $\geq 44\text{px}$.

8. PHASE 7: ANIMATIONS + DELIGHT (Hour 26–30)

8.1 The Delight Checklist

- ☐ Tree grows from root (staggered)
- ☐ Nodes pulse on hover
- ☐ Edges draw with `pathLength` animation
- ☐ Detail panel slides in with spring
- ☐ Stat cards count up
- ☐ Page transitions (fade + slide)
- ☐ Loading skeletons (not spinners)
- ☐ Empty states with illustrations
- ☐ Success toast on bridge completion
- ☐ Confetti on pathway achievement

8.2 Page Transitions

Create `web/components/layout/PageTransition.tsx`:

`tsx`

```
"use client";

import { motion } from "framer-motion";
```

```

export function PageTransition({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 8 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: -8 }}
      transition={{ duration: 0.3, ease: [0.16, 1, 0.3, 1] }}
    >
      {children}
    </motion.div>
  );
}

```

8.3 Loading Skeletons

Create `web/components/feedback/Skeleton.tsx`:

tsx

```

"use client";

import { cn } from "@lib/utils/cn";

export function Skeleton({ className }: { className?: string }) {
  return (
    <div className={cn("animate-pulse rounded-md bg-subtle/40", className)} />
  );
}

export function TreeSkeleton() {
  return (
    <div className="relative h-full w-full">
      <div className="absolute left-1/2 top-1/2 h-16 w-16 -translate-x-1/2 -translate-y-1/2 rounded-full bg-brand-500/20" />
      {Array.from({ length: 6 }).map((_, i) => {
        const angle = (i / 6) * Math.PI * 2;
        const x = Math.cos(angle) * 200;
        const y = Math.sin(angle) * 200;
        return (
          <div
            key={i}
            className="absolute left-1/2 top-1/2 h-10 w-10 -translate-x-1/2 -translate-y-1/2 rounded-full bg-subtle/40"

```

```

        style={{ transform: `translate(calc(-50% + ${x}px), calc(-50% +
        ${y}px))` }}
      />
    );
  })}
</div>
);
}

```

8.4 Confetti on Success

Create `web/components/feedback/Confetti.tsx`:

`tsx`

```

"use client";

import { useEffect } from "react";
import confetti from "canvas-confetti";

export function Confetti({ trigger }: { trigger: boolean }) {
  useEffect(() => {
    if (!trigger) return;
    confetti({
      particleCount: 80,
      spread: 70,
      origin: { y: 0.6 },
      colors: ["#8b5cf6", "#10b981", "#f59e0b"],
    });
  }, [trigger]);
  return null;
}

```

Add to `package.json`: `"canvas-confetti": "1.9.3"`.

Member 3's Hour 30 deliverable: The UI feels alive. Every interaction has feedback. Loading states are elegant.

9. PHASE 8: FINAL POLISH (Hour 30–33)

9.1 Accessibility Audit

- ☐

All interactive elements keyboard-accessible

- ☐
Focus rings visible
- ☐
ARIA labels on icon buttons
- ☐
`role="tree"` on the tree SVG
- ☐
`aria-selected` on selected nodes
- ☐
Color contrast $\geq 4.5:1$
- ☐
`prefers-reduced-motion` respected
- ☐
Screen reader text on stats

9.2 Performance Audit

- ☐
Lighthouse score > 95 (all categories)
- ☐
Tree renders in $< 500\text{ms}$ for 100 nodes
- ☐
Route-level code splitting
- ☐
Images lazy-loaded
- ☐
Fonts preloaded
- ☐
No layout shift (CLS < 0.1)

9.3 The 3-Minute Demo Script (UI Portion)

text

```
1:00–1:15 [Tree] "This is the student's academic tree.  
                Root is what they've learned.  
                Branches are what they can unlock."  
1:15–1:30 [Click node] "Click any node to see evidence."
```

1:30–1:45 [Pathways] "Three feasible pathways, side-by-side."

1:45–2:00 [Audit] "Click any decision to see the full audit trail."

Member 3's Hour 33 deliverable: Lighthouse > 95. Demo script rehearsed. Every page works.

10. PHASE 9: FALLBACK + HANDOFF (Hour 33–36)

10.1 Fallback Demo Video

Record a 3-minute screen capture of the full UI flow. If live demo crashes, play the video.

10.2 The Handoff Package

bash

```
tar -czf member3_artifacts.tar.gz \  
  web/ \  
  docs/ui/ \  
  docs/demo/ui-script.md
```

10.3 The Design System Document

Create docs/ui/DESIGN_SYSTEM.md :

markdown

```
# EduPathAI Design System  
  
## Colors  
- Brand: violet-500 → violet-700 gradient  
- Direct: emerald-500  
- Bridge: amber-500  
- Missing: rose-500  
- Review: slate-500  
  
## Typography  
- Display: Cal Sans / Inter  
- Body: Inter  
- Mono: JetBrains Mono  
  
## Spacing  
4px base. 8px rhythm.  
  
## Motion  
- Fast: 150ms  
- Base: 250ms  
- Slow: 500ms  
- Ease: cubic-bezier(0.16, 1, 0.3, 1)
```

Components

- StatCard: 4 variants (direct/bridge/missing/review)
- PathwayCard: 3 modes (fastest/balanced/max-preservation)
- Tree: radial (desktop) / vertical (tablet) / list (mobile)
- GapList: 4 gap types

11. THE COMMAND CENTER

bash

```
# 0. Setup
cd edupathai/web
source ../venv/bin/activate # for API access during dev
npm install

# 1. Dev server
npm run dev # http://localhost:3000

# 2. Type check
npm run typecheck

# 3. Lint
npm run lint

# 4. Build
npm run build

# 5. Preview production build
npm start

# 6. Test
npm test

# 7. Lighthouse
npx lighthouse http://localhost:3000 --view

# 8. Package
tar -czf member3_artifacts.tar.gz web/ docs/ui/
```

12. THE CREDIBILITY CHECKLIST FOR MEMBER 3

- ☐

Tree renders with root + branches.

- ☐

Tree is the first thing judge sees.

- ☐

Tree animates: nodes grow from root, edges draw.

- ☐

Tree is responsive: radial (desktop), vertical (tablet), list (mobile).

- ☐

Click any node → detail panel slides in.

- ☐

Detail panel shows evidence, missing outcomes, bridges.

- ☐

12+ pages built (dashboard, tree, pathways, gaps, audit, HEI, Gov, etc.).

- ☐

Dark mode + light mode.

- ☐

Mobile bottom nav works.

- ☐

Every interactive element is keyboard-accessible.

- ☐

Lighthouse > 95 (all categories).

- ☐

Loading states are skeletons, not spinners.

- ☐

Empty states have illustrations.

- ☐

Error boundaries catch failures.

- ☐

Animations respect `prefers-reduced-motion`.

- ☐

Colors meet WCAG AA contrast.

- ☐

No hardcoded values in components (all tokens).

- ☐ API types mirror `services/schemas.py` exactly.
 - ☐ Fallback demo video recorded.
 - ☐ 3-minute UI script rehearsed.
-

13. THE SINGLE LINE THAT DEFINES MEMBER 3

Member 3 renders the learner's academic future as a living tree — root is what they've learned, branches are what they can become — and ensures that every page, every interaction, every animation, and every responsive breakpoint tells the same story: this is a serious system that respects the learner's time, evidence, and dignity.

The Tree is the product. The pages are the story. The animations are the trust. The responsiveness is the respect.



EduPathAI
Ministry of Education

Gap-Find & BridgePath
Curriculum Reconciler

APAAR: 9081.****.2201 • B.Tech Computer Science & Engineering

Chain ID: global-inst-iitb-2026

● Chain Integrity: 100% Verified

Cryptographic Decision Bundle Explorer • Tamper-Evident Ledger

The sovereign audit trace ensures **immutable, tamper-evident verification** of AI and human academic decisions.



Tree View



Pathway
Solver



Gap
Analysis



Ledger



Profile



✓ **Identity Tokenized & Enrolled**
DigiLocker APAAR handshake

7f83b1657...

✎ **Multi-Model AI Extraction & Consensus** Consensus Score: 0.94
Gemini 2.0 Flash + DeepSeek
Consensus Score: 0.94 • v4.2 Prompt Engine Tue-04-31 17:35 AM

⚠ **Deterministic Recognition Gate** Status: BRIDGE_REQUIRED
Zero hallucination rule applied
Rule #7 matched: **Missing Network Flow Algorithms**
[Review Bridge Requirements](#)

👤 **Human Board Approval**
Signed by **BoS Convener Prof. S. Sen** via **Aadhaar e-Sign** ✓

decision_bundle.json

```
{
  "name": "metadata",
  "version": "0.0.0",
  "sha256": "4a7b9c2d57a9e3e9a268e9532a22beb0c",
  "ledgers": [
    {
      "sha256": "4a7b9c2d57a2d47ab964b77b2277a",
      "Solver parameters: {
        "presentation": "5",
        "times": 10,
        "masching": true,
        "parameter": false
      }
    },
    immutable S3 Object Lock pointers:
    "sha256": "1aa775e2e7a2c969d3737b1500507d9"
  ],
}
```

[Export Cryptographic Proof \(PDF\)](#)

[Replay Exact Decision State](#)

Audit Sync
Status



HEI Board of Studies pre-
approved bridge pathway

[Proceed & Update
Academic Plan](#)



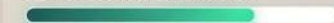


EduPathAI
Ministry of Education

**Gap-Find & BridgePath
Curriculum Reconciler**

APAAR: 9081.****.2201 • B.Tech Computer Science & Engineering

72% Content Alignment



Source: University of Calcutta (Data Structures) → Target: IIT Delhi (Advanced Data Structures & Algorithms)

**Recommended Bridge Courses
- BridgePath Engine**



Tree View



Pathway
Solver



Gap
Analysis



Ledger



Profile

Curriculum Outcome Delta

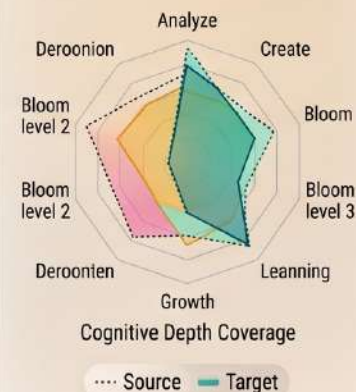
- | | | |
|-----------------------------|---|----------------------------------|
| 1. Memory Allocation | → | Dynamic Allocation |
| 2. Complexity Analysis | → | Amortized Complexity |
| 3. Linear Arrays & Lists | → | Advanced Sequences |
| 4. Trees & Simple Search | → | Balanced Trees & Red-Black Trees |
| 5. Search & Sort Algorithms | → | External Sorting & Hashing |
| 6. Basic Graph Traversal | → | Advanced Graph & Network Flows |

DIRECT MATCH (Bloom Level 3 - Apply)

GAP DETECTED: Missing Network Flow Algorithms

Bloom's Taxonomy Cognitive Depth Coverage

DIRECT MATCH (Bloom Level 4 - Analyze)



NPTEL: Analysis of Algorithms Unit 3 & 4

Duration: 18 Hours Online Lab Included

100% Outcome Coverage

Institution Approved

+ Add to Academic Plan

SWAYAM: Advanced Algorithmic Thinking Micro-Module

Duration: 12 Hours Self-Paced

+ Add to Academic Plan

Workload
Variance



HEI Board of Studies pre-
approved bridge pathway

**Proceed & Update
Academic Plan**





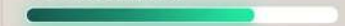
EduPathAI
Ministry of Education

Path-Solve Optimizer:
Multi-Objective Degree Pathway Generator

APAAR: 9081-****-2201 • B.Tech Computer Science & Engineering

IIT Bombay | Curriculum v2026.1

72% Recognized



Tree View



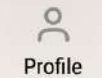
Pathway Solver



Gap Analysis



Ledger



Profile

Fastest Path

24 credits/sem

Minimize Graduation Time: 3 Semesters | high workload

Balanced Path

18 credits/sem

Optimal workload variance: 4 Semesters

Maximum Preservation

4 Semesters + 1 summer bridge

100% Credit Retention

Semester V - Spring 2027

18 Credits

Distributed Systems (CS-401)

4 Credits

Theory

Computer Networks

4 Credits

Theory & Lab

Operating Systems

4 Credits

Summer Bridge: Advanced Graph Algorithms (SWAYAM - 4 Weeks)

Semester VI - Fall 2027

16 Credits

Artificial Intelligence

4 Credits

Theory

High-Performance Computing

4 Credits

Lab

Software Engineering

4 Credits

Theory

Semester VII - Spring 2028

14 Credits

Natural Language Processing

4 Credits

Theory

Machine Learning

4 Credits

Lab

Interdisciplinary Elective

3 Credits

Theory

Semester VIII - Fall 2028

12 Credits

Capstone Project

6 Credits

Lab

Open Elective II

3 Credits

Ethics in AI

3 Credits

Workload Variance



Solver Confidence:
OPTIMAL (CBC 9.10)

Lock and Submit Pathway to Academic Council





EduPathAI
Ministry of Education

Student Academic Tree
& Competency Explorer

APAAR: 9081-****-2201 • B.Tech Computer Science & Engineering

IIT Bombay | Curriculum v2026.1

72% Recognized



Tree View



Pathway
Solver



Gap
Analysis

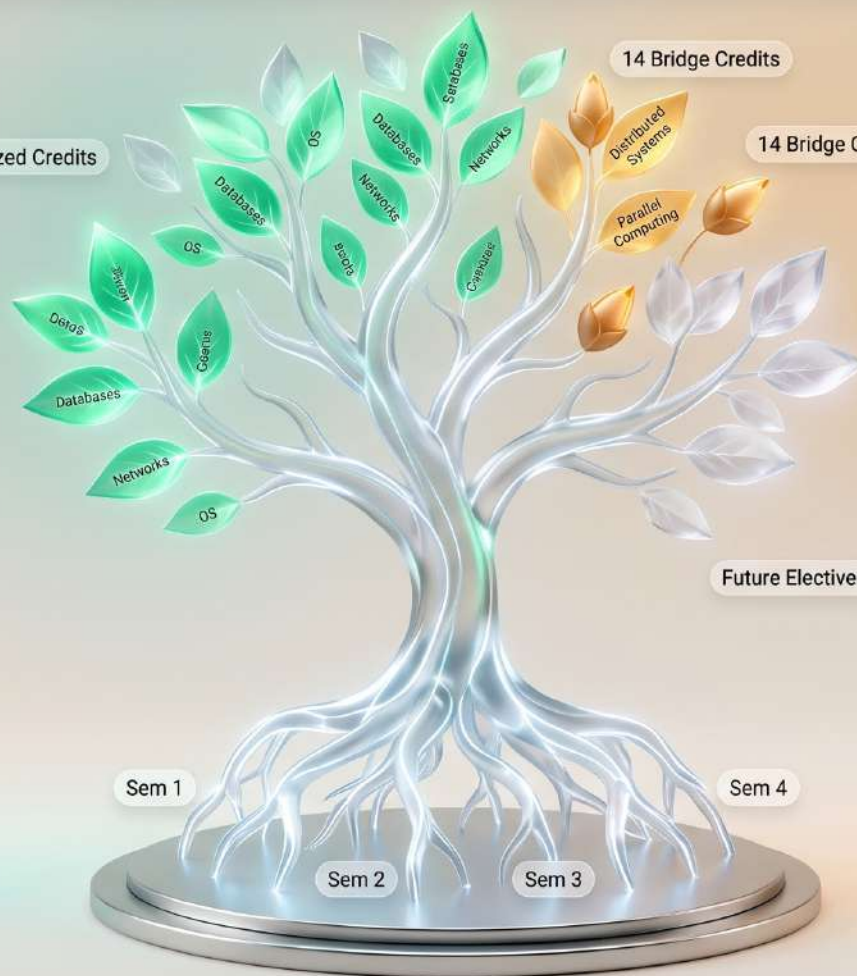


Ledger



Profile

48 Recognized Credits



14 Bridge Credits

14 Bridge Credits

Future Electives

Competency Node: Distributed Systems (CS-401)

BRIDGE REQUIRED
(0.78 Semantic Coverage)

Prerequisites & Alignment

- ✓ Direct prerequisites satisfied (3/3)
 - ✓ Computer Networks
 - ✓ Operating Systems
 - ✓ Data Structures
- Course Equivalency Gap:
Consensus protocols (Paxos/Raft)

[Enroll in NPTEL Bridge Course \(20h\)](#)



Identity Mode Selection

Learner / APAAR

Institutional BoS Reviewer

Ministry / State Nodal Officer

Access via DigiLocker & ABC ID

Enter 12-digit APAAR / ABC ID

Biometric
/ OTP ☐

Verify & Fetch

OR



Connect with MeriPehchan

hash-verified



DPDP Act (2023) privacy consent micro-card

• Zero PII persistence • Tokenized verification only

☐ Consent to secure data processing under DPDP Act (2023)



Intelligent Academic Mobility. Tamper-Proof Recognition.

The National AI-driven engine for seamless credit transfer, alignment, and authentication across India's higher education ecosystem.

14.2M

Credits Transferred



840+

HEIs Integrated



0.01%

Error Tolerance



RECOGNITION FRAMEWORK

Deterministic Solver



Cross-Institutional Recognition

Course A



Course B

✓ 96% Alignment

WORM Audit Ledger

SHA-256

